Rule Evolution for Formally Verifiable seL4 Applications

From Requirements to Execution with the Accountable Neuro-Symbolic SCP Workbench

ANONYMOUS AUTHOR(S)

Repairing a system model can invalidate the proofs, tests, and approvals for its generated code. The Spec–Code–Proof (SCP) Workbench supports model-based engineering from reviewed English requirements and design context through SysML v2 models and contracts, code generation and repair, formal checks, and seL4 execution. Supervised rule evolution turns tool diagnostics and reviewed repairs into versioned guidance with applicability conditions and required checks. Workers reuse selected rule files in isolated workspaces. The controller binds checks and approvals to reviewed revisions and retains costs across authorized repairs.

Held out of Monitor training, Producer/Consumer started from English and existing rulebooks with zero supplied target-model lines in the reported dispatch. Three successful build inventories total 26,758 source lines, including generated code, tests, and bundled support. In a later common-input Simple Network Guard (SNG) pair, the arm receiving a 66-entry rule bundle passed five model checks at 346,224 measured tokens. The baseline stopped at its repair limit after 388,726 tokens. The guided continuation delivered a 13,133-line build with 32 Verus proof units discharged under recorded premises and passing Rust and JVM tests. Source and image fingerprints link checked component code and the built Microkit image to 20.033 seconds of QEMU observation after readiness.

CCS Concepts: • **Software and its engineering** → **Formal methods**; • **Computing methodologies** → *Artificial intelligence.*

Additional Key Words and Phrases: supervised meta-learning, formal verification, neuro-symbolic engineering, seL4, scoped attestation, model-based systems engineering

1 Introduction

Behrang et al. [11] combine language models with program analysis to repair data races. Lee and Heo [29] derive formal test oracles from text to expose behavioral bugs. In embedded systems, changing a model contract can invalidate generated code, proofs, tests, and the approval under which they ran. An agent can use tool diagnostics to propose a repair. The engineer needs to know which obligations were checked for the revision, which assumptions the checks used, and who authorized the next step. These questions recur when a later system reuses lessons from an earlier repair.

We present the Spec–Code–Proof (SCP) Workbench, a neuro-symbolic system for model-based systems engineering (MBSE). It constructs SysML v2 models with GUMBO contracts from reviewed English and design context. The controller coordinates HAMR generation for Slang and Rust, checks with Logika and Verus, and builds and runs a seL4 application using Microkit (Figure 1) [15, 19, 38]. The controller supplies selected rule versions to workers in fresh workspaces. Acceptance requires evidence and approval for the current revision, assumptions, and scope.

Tahat et al. [45] used reusable rules and HAMR/Logika checks to generate and repair GUMBO contracts within supplied Isolette models. The companion demonstration supplied the Slang/Scala application logic [31]. The SCP Workbench extends this approach to constructing target models, checking the generated Rust and Slang implementations, and building and observing a runnable image. Its campaign record connects each stage to the revision it assessed and the decision that authorized continuation. Table 7 compares the starting artifacts and outcomes of related systems.

Our contributions are the following.

(1) **Executable engineering workflow.** A controller coordinates model proposals, generation, repairs, proofs, tests, builds, and runtime observation. It preserves earlier outcomes and cumulative expenditure across authorized continuations (Figure 3).

(2) **Budgeted supervised meta-learning.** Engineering attempts and tool diagnostics yield rules with applicability conditions, required checks, earlier versions, and supporting evidence. A declared file handoff carries selected rule versions into a new workspace, where subsequent attempts can refine them (Figures 4 and 5).
(3) **Scoped review and attestation.** Engineers delegate actions within approved limits and decide changes to assumptions, scope, and release. The protocol binds decisions to artifact fingerprints and evidence, and supports signed export for checking under an independently supplied policy (Figure 2).

The evaluation follows reuse across the Isolette Regulator holdout, Producer/Consumer, and the Simple Network Guard (SNG). Three successful build inventories contain 26,758 source lines (Table 3). In the later SNG comparison, the candidate receiving the 66-entry bundle passed all five model checks on shared task inputs. The controller rejected a proposal to delete an existing test and preserved its cost. The final Gate and Filter proof-input fingerprints match the corresponding Rust files in the build and observation inventories, and the loader digest identifies the image observed in QEMU [9].

2 Background and Running Example

2.1 Engineering and Verification Tools

SysML v2 models structure and interfaces [35]. GUMBO adds behavioral contracts [22]. An *assume–guarantee contract* (A, G) requires G when its assumptions A hold. An *obligation* is a property to establish under stated assumptions. HAMR derives infrastructure and contracts for Slang and Rust [21]. Logika checks Slang and connection obligations. Its configured integration checks use the cvc5 satisfiability modulo theories (SMT) solver [8, 18]. Verus checks Rust obligations and uses an SMT backend such as Z3 [48]. GumboX derives executable checks, and Microkit provides a seL4 image target [12, 41].

A shortened GUMBO example makes an ordering assumption explicit [8, 45]:

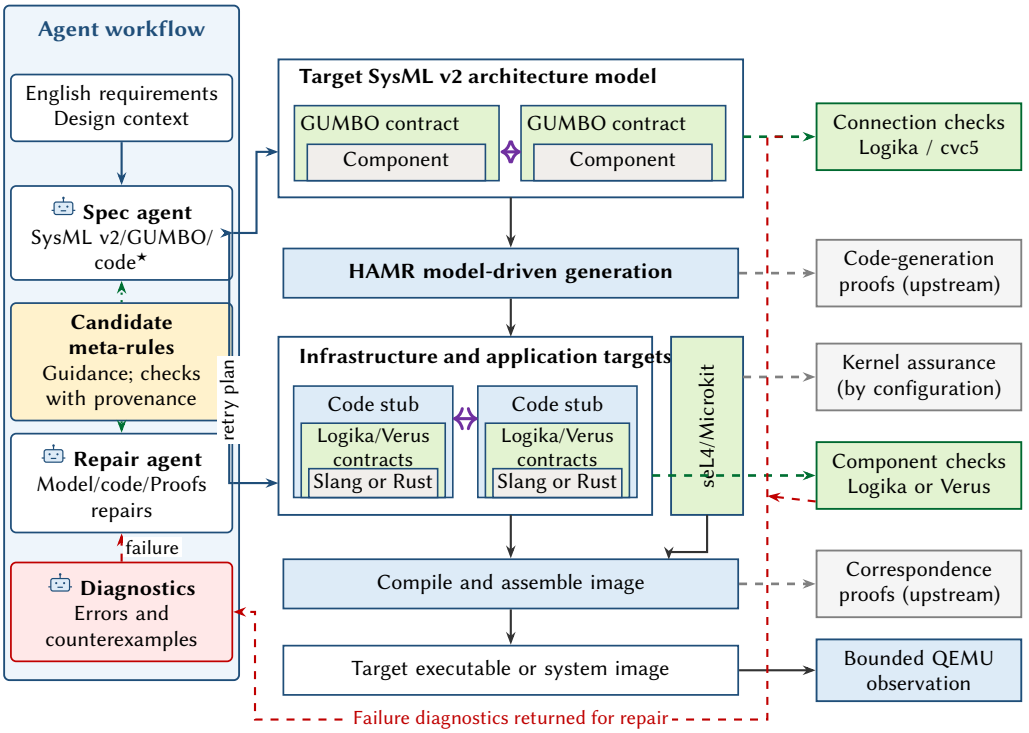
```
compute
  assume lowerLTUpper: lower < upper;
compute_cases
  case HEAT_ON:
    assume mode==NORMAL && temp<lower;
    guarantee heat_control==On;
```

Here `temp` is measured temperature; `lower` and `upper` are desired bounds; `mode` selects operation; and `heat_control` is the heater command. The global assumption orders the bounds; the case assumption selects when heating is required. Other declarations and cases are omitted.

2.2 A Missing Premise in a Reference Contract

Isolette regulates infant incubator temperature with a heater [20, 30]. Let θ be measured temperature, L and U the desired bounds, and h indicate heating. In NORMAL mode, the guarantees are $\theta < L \Rightarrow h$ and $\theta > U \Rightarrow \neg h$. The inspected fragment omitted the ordering premise and allowed contradictory demands [8].

The operator-interface assumption `EA-OI-6` requires the lower setting to be at least 1°F below the upper setting [30, Appendix A.3.4, p. A-11]. The recorded repair adds the reviewed premise $L < U$ [8]. Review considers representation, admissible ranges, and propagation. Section 5 follows the diagnosis and derived guidance.



Green callouts identify application checks. Gray callouts indicate upstream assurance where available. Verus uses Z3 for Rust proofs. GumboX supplies executable tests for the selected target.

Fig. 1. Engineering layers and repair feedback in SCP Workbench, adapted from Hardin et al. [19] and Tahat et al. [45, 46]. The star marks neuro-symbolic generation of SysML v2/GUMBO models and Rust/Verus and Slang/Scala artifacts through agent proposals, HAMR generation, and checker-guided repair. The nested boxes show components and their contracts. Red arrows return checker diagnostics for model, code, and proof repair. Table 2 records the checks exercised for SNG.

2.3 Learning from Engineering Experience

Recent research offers complementary ways to check proposals and learn from their outcomes. AWS's ARc uses human-reviewed policy formalizations to check natural-language claims with symbolic tools [3]. Google DeepMind's AlphaEvolve improves proposed algorithms using automated evaluators [34]. Google Research's WikiSkill retains execution experience in an external knowledge base for later skill revisions [47]. Anthropic's interpretability study identifies internal representations that language models can express in words and use in reasoning [17]. These examples motivate a workflow that checks a proposal, retains what the attempt taught, and makes that lesson available to the next task.

In-context learning uses examples and instructions supplied during a call [49]. Meta-learning changes the procedure used to approach later tasks [23]. SCP Workbench retains selected lessons across calls as written, versioned guidance. We call this step *verbalization*: it expresses reusable procedures and their applicability conditions from recorded requirements, attempts, diagnostics, and checks. Shared structure can guide the search under a finite allowance, and engineering tools assess the resulting proposals [1, 45].

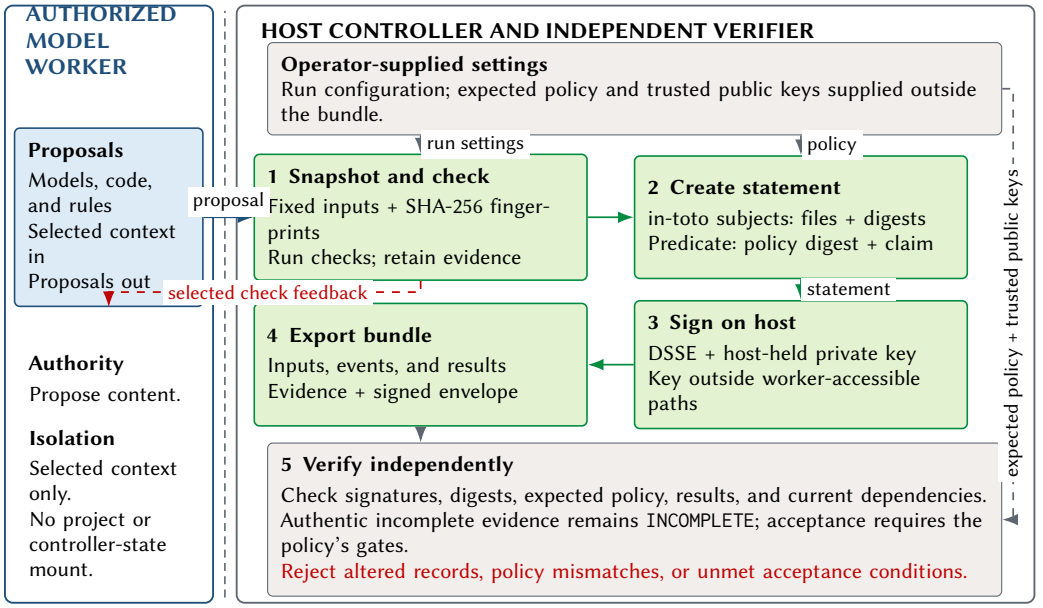


Fig. 2. Accountability boundary and optional evidence export in SCP Workbench [8, 25, 39]. The worker proposes artifacts; host services record checks and create a signed claim about selected evidence. Steps 1–4 prepare and export that record; step 5 checks it against independently supplied policy and trusted keys. A signature establishes the integrity and origin of the signed record. Local human authorization and acceptance of engineering obligations require their own recorded decisions and checks.

Figure 9 illustrates why early checks matter in embedded engineering. A mistaken formalization can propagate through placement choices, generated code, and proof attempts. Checking a choice at the stage where it is made can expose a local repair before generating its downstream artifacts. The workbench retains the failed check and the repair that followed, providing evidence for a lesson that can guide future attempts.

2.4 Evidence, Review, and Attestation

Evidence links a check to assessed files and assumptions. A *fingerprint* is a value computed from contents; comparing SHA-256 fingerprints detects changes when those contents are rechecked [8, 33]. Review records associate each decision with its reason, exact contents, and responsible reviewer identifier. An *attestation* states a claim about identified files. Its signature supports checking integrity and origin under a trusted key [25, 39]; Section 4 explains how these records support an audit.

3 Workbench Architecture and Accountability

SCP Workbench governs a target system's models, code, and proofs (Figure 1). The worker proposes artifacts; host services retain checks and decisions and optionally export signed evidence (Figure 2) [8].

3.1 Approved Scope and Campaign Records

At setup, the engineer approves the *scope*: the obligations, assumptions, required checks, and limits for the task. Acceptance requires every mandatory check in that scope to pass on current reviewed

contents with matching approval. Authorized exclusions retain their reasons and failed evidence without verification credit.

The algorithms use C for the *campaign record*: requirements and sources, artifact and rule versions, checks, decisions, scope, and resource consumption. A requirement table links source passages to their interpretation and evidence. Identifiers retrieve these records; fingerprints identify the exact contents checked or approved. Records also name the agent or human responsible for a proposal, check, or decision. Section 6 follows this review process, and Section 4 explains authorization and signed export.

3.2 Execution Environments and Shared Records

The execution service prepares inputs, invokes tools, and retains results. The engineer delegates generation, checks, and permitted repairs within approved limits on scope, time, tokens, and access to files and context. Changes to requirements, assumptions, or limits require a decision tied to the affected revision. The Python package exposes these services through the browser and command-line interface [8]. The measured Workbench snapshot contains 8,255 implementation lines across 51 Python files (Appendix A).

For example, `generate_and_check`(t, c) in `development.py` processes task t within campaign c, a group of tasks sharing resource limits. It checks approval and input versions, requests a budgeted proposal, prepares its workspace, and runs configured checks. The archive links each result to the task, model call, source inventory, command, and diagnostics. On continuation, the controller checks the current inputs and approval and carries prior charges forward.

Each case and comparison arm starts in a separate workspace. The host assembles the dispatch from declared task files and selected rule versions. An authorized continuation can pass the preceding candidate and diagnostics as inputs to a fresh worker. In the audited SNG runs the worker used an ephemeral Codex session, a temporary worker home, a cleared environment, allowlisted mounts, and selected context supplied through standard input. Tool network access was disabled. Formal checks use separate output directories, and common tool installations and caches can be shared. Runtime scripts build images in containers and launch configured QEMU guests. The host archive retains workspace identities, inputs, commands, outcomes, and observation scopes [6, 8].

These workspaces and their linked records also supply the experience used for learning. Verbalization turns selected requirements, attempts, diagnostics, and checked results into written lessons that can guide later work within the campaign or on another system (Section 5).

4 Scoped Review and Attestation

The campaign record identifies who proposed, checked, and approved a revision. Fingerprints bind those actions to exact contents; authorization checks govern protected actions, and signed exports support independent checking of the evidence.

4.1 Identify What Was Reviewed

Adding $L < U$ changes the heater model's fingerprint and requires review of the revised model and its new assumption. A decision covers identified task, rule, artifact, outcome, assumption, and scope records [8]. Before a protected action, the controller rechecks the reviewed contents and current approval. A mismatch makes the earlier decision inapplicable to the changed contents and returns the action for review. The history retains the responsible agent or human identifier, the decision, its rationale, and the affected versions. These links establish responsibility during an audit.

Local acceptance requires matching reviewed contents and dependencies, passing evidence for every included mandatory check, and current approval of that scope. A human can authorize partial

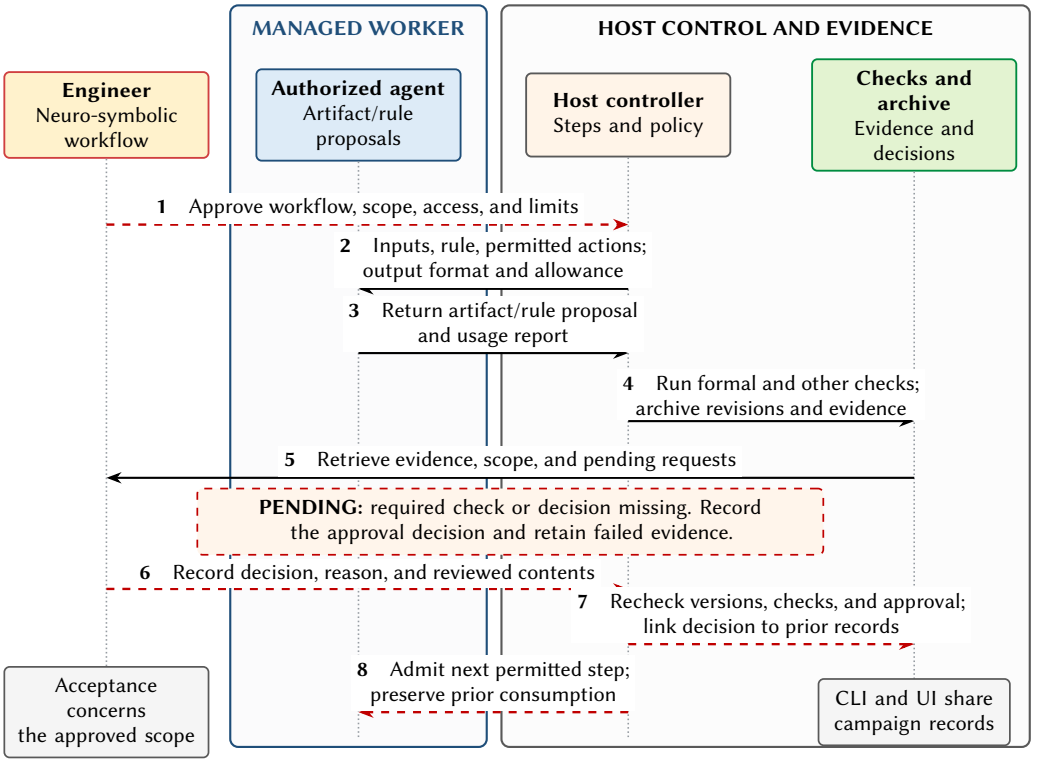


Fig. 3. Illustrative controlled handoff in SCP Workbench [8]. The eight numbered exchanges connect approved setup, bounded proposals, host checks, evidence retrieval, and recorded human decisions. The worker and host have separate responsibilities; a review request (5) and the recorded review decision (6) are separate records. Each subsequent task receives the selected rule revision and available allowance (2, 8), while prior outcomes and consumption remain linked to the case. Section 5.1 follows the same numbers.

release by recording exclusions and their reasons; deferred obligations remain required. Failed outcomes remain in the history. When the required checks and reviews pass, the task is complete within its approved scope. Algorithm 1 records these decisions and optionally exports a signed claim.

4.2 Signed Evidence and Independent Checks

Figure 2 follows five steps. **1–2. Check and describe.** The host snapshots inputs and records checks. An *in-toto Statement* names files and fingerprints as its *subjects*; its *predicate* states the claim and policy [25].

3–4. Sign and export. A *Dead Simple Signing Envelope* (DSSE) contains the statement, data-format label, and signatures covering both [39]. The *bundle* contains this envelope and supporting files. The private key remains outside worker-accessible directories.

5. Check independently. The receiver supplies the expected policy and trusted keys. The verifier checks signatures, file inventory and fingerprints, configuration, dependencies, and recorded results. Altered contents or inconsistent claims are rejected. Authentic evidence from unfinished work can yield `INCOMPLETE`; `ACCEPTED` also requires the recorded engineering-acceptance flag and passing evidence for the verifier's required engineering checks [8]. The signed record preserves the

Algorithm 1. Review a scope and check an optional signed export

Input: Campaign record C; available human decisions; optional export policy and trusted keys.
Output: Updated C; local review outcome q; export verdict v.

Record decisions about the current scope
Refresh evidence and scope; set q to PENDING
foreach available decision **do**
  if a grant or deferral concerns changed contents **then**
    Set q to STALE; stop this loop
  if acceptance is rejected **then**
    Record the rejection; set q to REJECTED; stop this loop
  Apply authorized scope changes, deferrals, or revocations; preserve original outcomes
  if the human decision grants acceptance **then**
    Recheck the subject fingerprint and mandatory scoped checks together; approve only if they pass
  Refresh the campaign record and the contents under review
if required contents have changed **then**
  Set q to STALE
if q is neither stale nor rejected **then**
  Set q to accepted only if every mandatory check in the approved scope passes and its approval is current
Check a signed export, when requested
if no export is requested **then**
  return q, NOT EXPORTED
Package selected run evidence and sign it on the host
Check the signature, contents, and claimed result against the external policy and trusted keys; obtain v
return q, v

identity of the exported evidence, while the local review records preserve approval and responsibility for the declared scope.

5 Budgeted Supervised Meta-Learning

The handoff in Figure 3 supplies reviewed tasks with selected guidance. The learning process in Figure 4 uses attempts and checks from campaign workspaces to revise that guidance. It repeats within a campaign and carries accumulated lessons into later systems. The heater case shows how a failure can motivate a new rule and define where it applies.

5.1 Controlled Handoff

A *handoff* supplies an authorized agent with inputs and permitted actions and returns its proposal for host checks. Figure 3 follows eight exchanges.

1–2. Approve and prepare. The engineer approves the workflow, scope, access, and limits (1). The harness retrieves current inputs and the selected rule, checks allowance, and specifies permitted actions and output format (2). Evaluator-only artifacts remain separate from dispatched context. Shared task documents and approved reference-derived supplements are declared inputs whose provenance is retained.

3–4. Propose and check. The worker returns an artifact or rule proposal and usage report (3). The host runs formal and other checks and records versions, outcomes, and fingerprints (4).

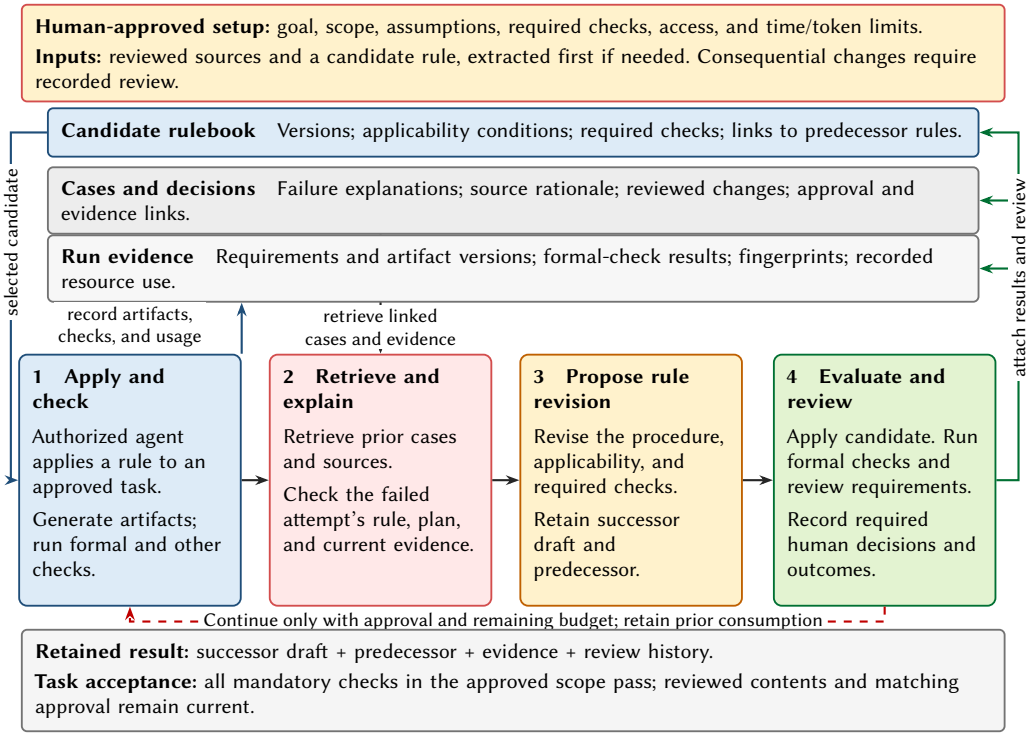


Fig. 4. Supervised rule evolution in SCP Workbench. A layered presentation inspired by WikiSkill [47] separates recorded evidence, cases and decisions, and candidate guidance. The four stages apply a candidate, interpret prior experience, propose a revision, and evaluate it under review. Updates retain the preceding rule and supporting records. Further attempts require current approval and remaining allowance. Acceptance concerns the reviewed task scope; candidate rules require independent tests before publication for reuse. Section 5.2 explains the stages; Figure 5 gives a concrete rule history.

5–7. Retrieve and review. The engineer retrieves pending requests and evidence (5), records the decision and rationale (6), and the host checks its contents and links it to prior records (7). A request alone grants no authority.

8. Continue within limits. Agents select permitted repairs while approval and allowance remain current. Expanding scope, assumptions, access, or budgets requires renewed human authorization. Prior charges and unresolved outcomes remain recorded [8].

A revised rule selected for another task enters at exchange 2; its outcomes return at exchange 4. Figure 4 explains how this experience informs revision.

5.2 Supervised Rule Evolution

Figure 4 connects run evidence, cases and decisions, and candidate rules through four stages governed by the handoff.

1. Apply and check. Extract an initial candidate from reviewed sources when needed. Apply the selected rule to an approved task, run formal and other checks, and retain versions, outcomes, and resource use.

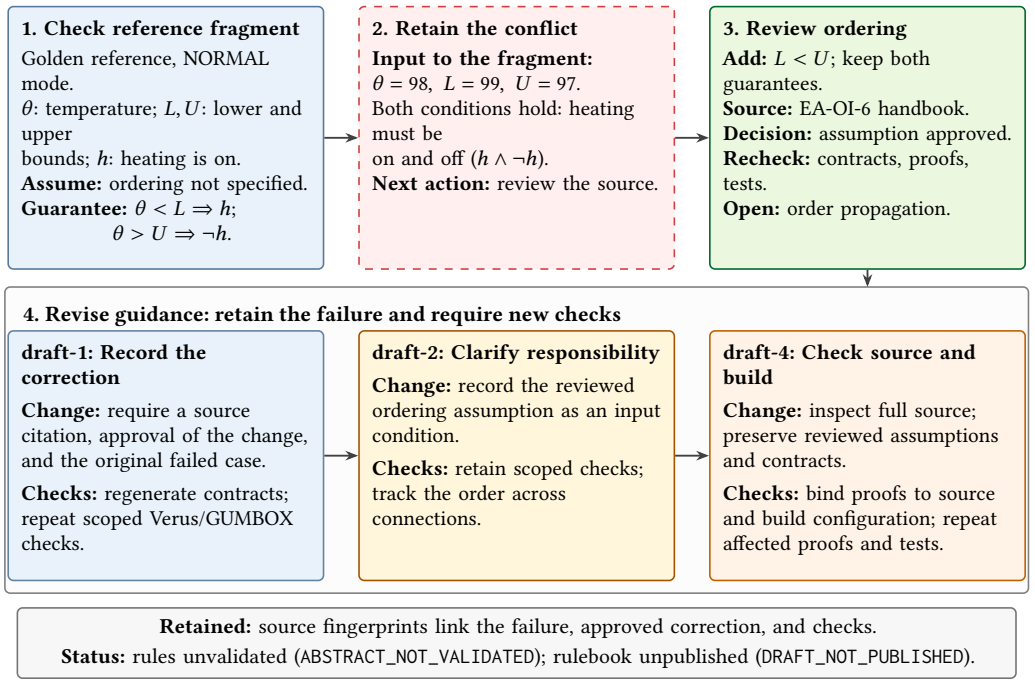


Fig. 5. A heater reference case becomes a source of candidate guidance [8]. Steps 1–2 expose conflicting guarantees in the contract fragment. Step 3 records review of an added ordering assumption and the checks required afterwards. Step 4 shows selected rule revisions with links to sources and checks; drafts 1, 2, and 4 are selected from the archive. The added assumption was approved, while the candidate rules await qualification. Horizontal arrows show revision order; intermediate and later drafts are omitted.

2. Retrieve and explain. Retrieve the attempt, source context, and review history. Express the reusable structure, procedure, or concept suggested by that experience. Refining a rule requires current failed evidence from that candidate and its approved plan.

3. Propose a revision. State the revised procedure, the tasks it applies to, and the checks it requires. Retain the preceding rule and supporting evidence.

4. Evaluate and review. Apply the candidate with fresh checks and requirements review. Record consequential decisions separately. Acceptance concerns the approved task scope; publishing a reusable rule requires independent qualification.

Missing evidence, approval, or allowance leaves work pending. Continuation retains earlier consumption and remains part of the same workflow. Approved exclusions retain failed outcomes without verification credit [8].

5.3 From Experience to a Candidate Rule

Learning turns recorded attempts, diagnostics, and checks into reusable guidance. A *meta-rule* states when it applies, its assumptions, a procedure, and the required checks. The *Generalizer* extracts a procedure shared by related tasks; the *Specializer* supplies the current interfaces, assumptions, and obligations. These strategies use the same model worker. Each proposed rule retains its sources and earlier versions, extending SCP's supervised learning and repair process with persistent review and accounting [8, 45].

The file handoff in Section 3.2 makes the reusable knowledge explicit. A new target receives a rule snapshot with its own task inputs. The host archive retains the diagnostic, rule revision, and check that motivated each lesson, so a subsequent attempt can refine the rule and leave the earlier history inspectable. Section 7 identifies the rules carried between the evaluated cases.

Earlier SCP used a similarity comparison when considering formalizations [45]. The workbench computes cosine distance to retrieve related rules and to compare contract features after architecture checking. It also records model forecasts with explanations. These signals can help prioritize interpretation and review (Section 6).

An early *semantic gate* would use the predicted agreement between English requirements and a proposed interpretation to choose whether to proceed to formalization, refine the interpretation, or request review. Automatic admission and branch ranking are proposed extensions of the available comparison and forecast services. Formal checks and human review then assess the candidate. Appendix D gives the representations, cosine geometry, and connection to budgeted candidate selection [16, 46].

5.4 Learning from the Heater Case

Figure 5 follows a reference correction (upper row) and the candidate guidance derived from it (lower row).

(1) Check the reference contract. Using the notation of Section 2, the heater fragment requires $\theta < L \Rightarrow h$ and $\theta > U \Rightarrow \neg h$ in NORMAL mode, without ordering L and U.

(2) Retain the conflicting input. The diagnostic script uses $\theta = 98$, $L = 99$, and $U = 97$: both conditions hold, requiring heating on and off simultaneously [8]. This exposes the missing constraint in the fragment; $U = 97$ lies outside the handbook's full upper-bound range [30]. Giving one output priority changes a guarantee; filtering tests changes coverage; adding an ordering assumption restricts inputs. Each repair therefore needs an explanation of how it meets the requirement.

(3) Review the assumption. The repair record approves $L < U$ and retains both guarantees. It cites `EA-OI-6`, the operator-interface environmental assumption in Lempia and Miller's handbook requiring $L \leq U - 1°\mathrm{F}$ [30, Appendix A.3.4, p. A-11]. Strict ordering gives this margin for integer degree settings; other representations require the margin to be stated explicitly. Review records responsibility for the chosen premise; proof and tests assess the repaired artifact, with internal propagation retained as an obligation.

(4) Revise the rule. The figure shows revisions of `GOLDEN-DEFECT-CURATION-001`. `draft-1` requires cited correction, consent, failure retention, regeneration, and checks with Verus and GumboX. `draft-2` distinguishes environmental assumptions from software guarantees. `draft-4` adds the complete sources and build context and requires reassessment of regenerated contracts and proofs. Each revision retains its predecessor and supporting evidence; these archived candidates await qualification for publication [8].

Section F.1 explains retrieval of these linked records.

5.5 Learning Within an Allowance

Algorithm 2 receives the case, reviewed sources, and an optional candidate selected directly or through the retrieval comparison. It can also extract a new draft from the sources. Algorithm 3 supplies the development and fidelity-review procedure. *Reading the procedures.* A *budgeted model step* reserves an allowance, calls the model, records usage, and retains the outcome. It returns PENDING if another check, decision, allowance, or human instruction is needed. A changed input produces STALE, which requires renewed review. Every return keeps the campaign history. Section 7 reports a failure in retaining a detailed call record; the procedures describe the intended order of operations.

Algorithm 2. Learn a candidate rule from retained campaign experience

Input: Campaign record C; reviewed sources M; optional draft rule r.
Output: Updated C and r; current task outcome q.

1 **Prepare the rule**
2 **if** the source or plan is unapproved **then**
3 **return** PENDING for review
4 **if** the reviewed source or plan changed **then**
5 **return** STALE
6 **if** no draft r is supplied **then**
7 Extract r from M with a budgeted call
8 **if** no draft is returned **then**
9 **return** PENDING
10 Retain r, its sources, and the call record
11 **Apply and revise the rule**
12 **while** recorded permission and budget permit another attempt **do**
13 Select a currently approved prepared task T
14 **if** no such task is available **then**
15 **return** PENDING
16 Apply r to T in a budgeted call; check; set q
17 **if** q is not `NOT_VALIDATED` **then**
18 **return** q
19 **if** the failure uses another rule or plan **then**
20 **return** REJECTED
21 **if** diagnostic inputs changed **then**
22 **return** STALE
23 Revise r from permitted feedback in a budgeted call
24 **if** no revision is returned **then**
25 **return** PENDING
26 Retain both rules, failure, and attempts; use the new rule as r
27 **return** PENDING for more allowance or human guidance

The first phase extracts a draft and retains its origin (lines 3–10). The second applies it to a reviewed task. Rewriting requires current failed evidence from the same rule and plan, recorded as `NOT_VALIDATED`; the revised rule keeps links to the failure and earlier rule (lines 13–26).

Resource accounting. The campaign's resource ledger retains time, tokens, and attempts across retries. Process deadlines bound execution; token reservations estimate usage, then the ledger records measured consumption and any excess. The estimate may be lower than the usage reported by the provider. Exhausted allowances or unresolved usage stop calls. Approved extensions preserve prior consumption (Section F.1).

Qualification for reuse. Algorithm 2 returns a task outcome and a revised candidate. Publishing that rule uses the separate qualification and approval service, which checks evidence for declared examples. The archived heater drafts retain their qualification status [8].

6 Requirements Fidelity and Persistent Review

Requirements fidelity means that the checked obligations preserve the reviewed engineering intent. Appendix C illustrates the review and refinement loop. The reviewer reads the English source line by line and builds a requirement table: each obligation has a source passage, an interpretation, its intended model or code representation, and the assumptions needed to check it. Ambiguities, omissions, and proposed changes remain visible for review. This review includes checking extracted text against its source before approving the input.

6.1 From English Requirements to Checked Evidence

Semantic assessment examines whether the interpretation and formalization express the requirement. For the heater, the reviewer must decide whether the premise $L < U$ represents the intended environment. Model explanations and comparisons can help identify questions for review. Contract-feature similarity helps locate related obligations; logical equivalence requires an appropriate logical check [3, 45]. Our adaptation of CLOVER connects English requirements, semantic assessment, and verification evidence around the formalization [42]. The proposed predictor would estimate agreement with reviewed intent and the time and tokens for the next attempt. It would use prior reviews and comparable check records to prioritize candidates under the remaining allowance (Appendix D).

The reviewed task specifies inputs, rule revisions, required checks, and limits. A budgeted proposal proceeds through HAMR generation and the configured checks. Parsing checks syntax and types; architecture comparison checks components and connections; proofs establish contracts under their premises. Tests and runtime observations cover their configured cases and periods. The requirement table connects these results to the original obligations and records unresolved questions. Repairs within approved scope continue the same trajectory with its earlier consumption. Changes to requirements, assumptions, scope, or limits return to human review.

6.2 Close the Loop with an Identified Decision

Each campaign record retains its English inputs, candidate revisions, checks, costs, and decisions. Requirement tables and case audits connect the reviewed intent to model contracts and evidence. Fingerprints identify the exact contents covered by a review. Rechecking detects changed contents and triggers reassessment of the affected claims. Each assessment or decision identifies the responsible agent or human reviewer.

A task is complete when every required check and review in its declared scope has passed. The human owner decides release. An approved partial release states the included scope, exclusions, assumptions, and remaining obligations; excluded obligations receive no verification credit. The acceptance conditions in Section 4 bind the decision to current contents and evidence.

The case studies illustrate this process. Isolette source review recovered omitted environmental dependencies and identified timeout and timing discrepancies. Producer/Consumer and SNG audits connected source requirements to model contracts and proof evidence. These reviews establish which interpretations and assumptions the evidence supports. Appendix C gives Algorithm 3; Appendix E follows one SNG requirement through its contracts, proofs, and assumptions.

7 Evaluation

We evaluate guidance reuse and reconstruction (RQ1), control of repairs and continuation (RQ2), and the connection between reviewed requirements and checked artifacts (RQ3). The cases follow guidance from Isolette Monitor to the Regulator holdout, then examine new construction and

later reconstruction for Producer/Consumer and the Simple Network Guard (SNG). We distinguish model checkpoints, successful builds, and bounded runtime observations.

Protocol and timing. Candidate workspaces isolate runs of the INSPECTA tools [19]. Experiment calls requested `gpt-6-astra` through Codex 0.155.1. In each later reconstruction pair, K receives the common requirements, libraries, and guides, including the shared design supplement where applicable. K^+ adds a selected rule bundle. Each arm has a 500,000-token allowance and a 24-minute model-stage limit. Each pair reports one trajectory per arm. Completion requires all five specified model checks. Table 1 reports the measured work through that endpoint or the observed stop [13]. Appendix B gives the configuration, clock definitions, and estimated debits.

Table 1. Later common-input model pairs. Times are in seconds. Dispatch includes client and controller call handling, checks sum the recorded tool intervals, and elapsed time includes gaps. A dash marks an unavailable elapsed interval at that endpoint.

Case and arm	Outcome	Measured tokens	Dispatch	Checks	Elapsed
Producer/Consumer K	HTTP 429	48,236	36.35	17.40	57.86
Producer/Consumer K^+	Five checks passed	179,515	109.53	94.45	210.19
SNG K	Repair limit	388,726	682.71	362.30	1,052.77
SNG K^+	Five checks passed	346,224	532.43	154.10	—

7.1 Isolette: Monitor Guidance and Regulator Continuation

Isolette regulates an infant incubator [20]. Monitor supplied training examples and Regulator served as a component holdout for model generation and extension to Rust/Verus. Six successive candidates used draft guidance from preceding attempts. The fifth first passed parsing and architecture capture. Revised, approved English and reference abstractions then supported structural agreement [5].

The revised generation completion was recorded 22 min 27 s after revised-input approval. The historical ledger attributes 341,178 measured tokens and 32.43 minutes of summed reservation-to-usage intervals to generation attempts and English preparation, with a separate 1M estimated timeout debit. The repaired Rust continuation completed 11 of 11 component proof/test pairs with 62 conditional obligations. Twenty-nine GumboX and 23 other test functions passed. A Microkit build passed, followed by a 20.008-second observation command including startup. The results cover the approved environmental premises and physical-operator exclusion. Appendix A gives the review chronology and separately scoped probes.

7.2 Producer/Consumer: First Construction and Later Reconstruction

First construction. Producer/Consumer was held out of the documented Monitor training split [8]. The campaign team reports that the first separate workspace received English requirements and existing rulebooks. The worker constructed the SysML v2/GUMBO target from scratch. Its repairs became a source of guidance for later Producer/Consumer and SNG work [4]. A separate historical checkpoint reports 2 of 2 component pairs, 5 of 5 GumboX tests, and 4 of 4 other tests passing on 23 September, with bounded observation. The case-level summary leaves the checked candidate's identity unspecified [6].

Later paired reconstruction. The producer's output range $[0, 90]$ implies the consumer's assumption $[0, 100]$. Both arms received identical English, 15 libraries, nine workflow guides, and the same controller. K^+ ran first with six general entries from Isolette/HAMR and three from earlier

Producer/Consumer repairs covering port refinement, effective processor binding, and signed-32 literals.

The five checks cover parsing and types, JVM generation, the Logika connection implication, architecture capture, and effective binding. K^+ passed the checkpoint and a separate directed-graph comparison. The paired K stopped on HTTP 429, as did a separate authorized follow-up. The paired row describes work observed before interruption, and K's completion time and token use remain unknown. Appendix B reports the follow-up timing and the separate 1M estimated debit for each interruption [4].

7.3 SNG: First Construction and Later Reconstruction

SNG passes classified messages through a gate and filter to a receiver, with contracts for dropping, forwarding, and payload bounds [38].

First construction with earlier-system rules. All 11 retained entries at the first guided dispatch came from general rules and earlier system work, including Producer/Consumer repairs. Both arms received SNG English and guides with SNG examples. The guided arm also received approved reference-derived design choices. It passed its in-loop model checkpoint with two verified connection units after five repairs and 397,520 measured tokens. The ordinary-input arm stopped after six repairs and 387,144 measured tokens. The differing design inputs limit attribution to guidance (Appendix A).

Later reconstruction with shared design inputs. The earlier SNG attempts and recoveries yielded 11 draft SNG lessons. A separate reconstruction supplied K^+ with 66 selected entries, including those drafts and earlier-system guidance. Both arms used the same English, libraries, guides, approved design supplement, checks, and allowance. The five checks cover parsing and types, Microkit generation, JVM generation, Logika integration, and architecture capture. The guided arm passed all five and a separate directed-graph comparison. It reached the checkpoint with 42,502 fewer measured tokens than K used at its repair-limit stop. Dispatch plus check intervals summed to 686.53 s for K^+ at the checkpoint and 1,045.02 s for K at the stop. The endpoints differ, and the K stop leaves time to completion unknown. This pair evaluates the full supplied bundle (RQ1).

With the five-check checkpoint as the target, K^+ completed 1/1 at 346,224 measured tokens per completion. K completed 0/1 under the same 500,000-token ceiling. Appendix B uses $+\infty$ to display zero completed targets and retains the measured tokens at the stop.

One retained lesson came from a parser rejection. An earlier SysML model used rendered Slang literal syntax. The draft rule distinguishes SysML's `0 [s32]` from the generated Slang notation. The archive links the rejected attempt to the lesson and matches its text in the supplied 66-entry snapshot (Appendix A).

Continuation of the later candidate. A source-preservation check rejected a proposal to remove an existing test function. The controller retained its 78,469-token charge, bringing usage to 424,693. Authorized paid repairs added 151,909 tokens. Two later authorized repairs addressed serialization and initialization without additional experiment-model calls. The guided trajectory used 576,602 worker tokens and 18.72 minutes of dispatch across eight calls. From guided controller start to observation end it took 2 h 31 min 21 s, including approval pauses. The K arm's dependent checks were `NOT_RUN`. Table 2 reports the guided build and checks.

From requirement to image. The SNG requirement `Req_R_2.a` clamps a Restricted payload above 100 to 100. The audit follows its GUMBO guarantee into two Verus-checked contracts. Gate preserves the Restricted message, and Filter requires a present Restricted output with payload 100. At the model layer, a Logika check establishes that Gate's guarantee implies Filter's assumption that

Table 2. Artifact provenance and evidence for the later SNG K^+ trajectory [9].

Stage	Produced artifact	Check or observation
Model	SysML v2/GUMBO from the declared English, guides, and design inputs.	Five model checks and directed-graph comparison passed.
Code and proofs	HAMR-generated infrastructure and contracts with repaired Rust and Slang application artifacts.	Verus verified 32 target-crate units, including six initialization/composition checks. Six English component relations were checked under recorded premises.
Tests	Rust and Slang/JVM test results.	33 Rust test functions and 808 JVM test cases passed.
Image	4,957,232-byte Microkit loader image.	Verification-enabled build passed in 103.501 s. Build and observation inventories matched 201 paths.
Runtime	QEMU loader run and receiver log.	20.033 s after readiness, 18 receiver messages, zero configured anomaly matches, and one interrupt-controller warning.

no Critical message arrives, provided the connected values agree. The dispatch proofs assume empty output slots at entry. Factory and composition proofs support that premise for the reviewed initialization path. The recorded Gate and Filter proof inputs match the fingerprints of those Rust files in the build and observation inventories. The loader digest identifies the QEMU image, and the runtime log records boot and receiver activity. The clamp result rests on the scoped contracts and their premises. Wrapper and event-buffer correspondence, transport, non-interference, and physical timing remain open [40]. Appendix E gives the exact contracts, proof records, and premises.

7.4 Artifact Size and Completion Time

The three successful build outputs contain **26,758 source lines**. Table 3 separates these builds from the earlier model endpoints so their sizes and recorded times can be interpreted together. Appendix A gives per-language counts for the builds and the additional Scala inventories.

Table 3. Output inventories and recorded timing intervals. Counts include contracts, proofs, generated code, tests, and bundled support in the listed languages. Ordinary comments and blank lines are excluded. Dashes mark unavailable measurements.

Case and endpoint	Lines	Files	Recorded interval
Isolette revised model generation	—	—	22 min 27 s from input approval to generation completion record
Isolette Rust/Microkit build	12,363	177	Observation command 20.008 s including startup
Producer/Consumer K^+ JVM checkpoint	9,355	76	210.19 s elapsed to the five-check endpoint
Producer/Consumer Rust/Microkit resumption	1,262	34	—
SNG K^+ Rust/Microkit and JVM build	13,133	151	2 h 31 min 21 s from guided controller start through observation, including pauses
Sum of three builds	**26,758**	**362**	

Dedicated tests, harnesses, test support, and test-data generators account for 4,974 lines in the Isolette build, 537 in the Producer/Consumer Rust resumption, and 5,575 of 13,133 lines (42.5%) in the SNG build. SNG includes 1,930 lines of tests, harnesses, and their support and 3,645 lines of test-data generators. The two core component files per backend contain 291 Rust and 92 Slang lines, respectively, including embedded contracts. The 808 passing JVM cases count executions. Appendix A.1 gives the file-role breakdown for each backend.

7.5 Model-Token Costs and Authorization

For total input I, cached input H, and output O, GPT-6 Astra Standard base pricing gives $(10(I - H) + H + 50O)/10^6$ USD [36]. Table 4 separates the worker trajectory from the supporting sessions. Those sessions span preparation, both arms, and continuation. They recorded 131.61M tokens, of which 129.84M were cached input. At base rates cached input contributes $129.84 to their $166.67 estimate. Allocating both later arms and those sessions to the 13,133-line build gives $179.06 at base rates. Pricing all supporting usage at long-prompt rates instead raises this allocation to $333.75, or $25.41 per 1,000 inventoried lines. Appendix B gives the derivation and rate scenarios.

Table 4. Illustrative SNG base-rate model-token costs. The K^+ model checkpoint is included in the guided continuation. Per-line rates use the delivered source inventory. Tools, hosts, and labor are outside the model-token estimates.

Scope	USD	USD per 1,000 lines
K model-stage stop	4.76	—
K^+ model checkpoint	4.25	—
K^+ worker through observation	7.64	0.58
Three supporting sessions	166.67	—
Later pair and three supporting sessions	179.06	13.63

SNG's continuation raised the guided worker allowance from 500,000 to 600,000 tokens. The runtime authorization shortened the planned observation from 320 to 20 seconds and extended the deadline by one hour. Earlier consumption and eight used repairs stayed in the record. On 8 November the owner approved release of the three case results within their recorded scopes. The receipts identify reviewed contents, premises, and exclusions (Appendix A).

Experiment-worker ledger. Table 5 partitions 117 distinct calls through SNG closeout, recorded at 10:23:50 UTC on 6 November 2026. Each row combines that case's attempts and preparation phases. Measured tokens equal input plus output; cached input is a subset of input. Seven interrupted calls carry estimated debits. Tables 1 and 8 identify the selected task costs within these histories.

Table 5. Experiment-worker accounting through SNG closeout. Each call appears once. Historical acquisition in Table 16 is included in these rows.

History	Input	Output	Cached input	Measured	Est. debit
Isolette	702,356	100,364	37,888	802,720	1M
Producer/Consumer	1,071,521	34,521	46,592	1,106,042	2M
SNG	1,635,512	114,480	23,552	1,749,992	0
Shared	589,315	68,692	13,056	658,007	4M
Total	3,998,704	318,057	121,088	4,316,761	7M

The total is 4,316,761 + 7,000,000 = 11,316,761 accounted tokens. The overall experiment-worker allowance was 28M tokens. The SNG iteration used 965,328 measured tokens against a separate 1M

limit, covering the stopped baseline and guided work through observation. Appendix B partitions earlier acquisition and nested campaign accounting.

7.6 Evidence Retention and Scope

The Producer/Consumer source audit mapped 36 model/scope items and two workflow constraints from the English requirements. Review criteria were recorded after the K^+ run completed and before evaluator inspection. The K^+ audit recorded 34 represented items and four satisfied items, with one verified connection implication. The SNG audit followed observation and linked six English component relations to contracts and proofs under recorded premises. Appendix A gives the review chronology. Appendix E traces named requirements and their assumptions.

Controller tests. The controlled tests used fresh stores and controlled time. The audit matched 45 saved assertions across P01–P12. Table 6 reports their results and a fault probe for metering failure (RQ2–RQ3). F01 injected a 100k estimated debit against a 200k reservation. The resulting exception left the reservation and unresolved-usage state in place. The controller blocked further calls but saved no detailed call record. The control with a matching 200k debit retained the record.

Table 6. Controlled tests of the controller. F01 and its control exercise the reservation-to-debit path.

Test group	Recorded result
Budget	7/7 cases passed.
Authorization	3/3 cases passed.
Durability	1/1 case passed.
Retrieval	1/1 case passed. Decisions and evidence recovered after normal service reopening.
F01, 200k reservation and 100k debit	Exception. Further calls blocked. No detailed call record saved.
Control, 200k reservation and debit	One detailed call record retained.

The application checkpoint exercises revision binding in fixture-based tests. Changed evidence blocks approval, a changed Rust file or Cargo configuration invalidates verification credit, and a tampered signed subject is rejected [7, 8]. The SNG continuation supplies an engineering example with a rejected proposal, a separately authorized scope, and matched build and runtime receipts.

Inspecting and repeating results. The archived records support inspection of the input snapshot, decisions, and observed endpoints. Replaying SNG would start from the frozen implementation and image with the recorded scope held fixed. Fresh synthesis starts from the English and selected rulebook and can produce a different candidate requiring its own review. Appendix A lists the records required for replay.

8 Related Work

Table 7 compares starting artifacts, checks, and endpoints. Lee and Heo formalize intent for test oracles, Huang et al. connect symbolic checks to robot execution and reuse, and Zhang et al. validate project-scale translation. SCP Workbench keeps a system model, generated code in different languages, and an executable image aligned across repairs. Its archive associates each layer's result with the checked revision and the decision that allowed work to continue. Cibrián et al. validate generated SysML v2 models [14]. Research Preference Models motivate the proposed candidate-ranking policy in Appendix D [16].

Learning from analyzers and proof checkers. Getafix learns contextual repair patterns for static-analysis findings [10]. Passerine evaluates LLM-guided repair on industrial bug reports [37]. Baye-Smith learns probabilistic models that use feedback to prioritize static-analysis alarms [27]. Cobble-stone reuses successful parts of proposed Coq proofs and iterates on the remaining goals [26]. These techniques offer complementary ways to propose repairs, prioritize diagnostics, and discharge obligations. SCP Workbench coordinates feedback across changes to contracts, generated code, and executable images, and retains the evidence and authority for each repair.

Experience as reusable guidance. WikiSkill distills execution experience into persistent knowledge and revisable skills [47]. This is close to our use of recorded diagnostics to derive written rules. SCP Workbench makes the handoff between workers explicit through selected rule files and declared task inputs. It links each rule revision to tool results and human decisions across model, proof, test, and execution stages. The shared design question is what to retain from an attempt. Our engineering cases also ask which checks and approval a reused rule must face on the next system.

9 Limitations and Future Work

The reported handoffs demonstrate reuse in the observed engineering tasks. The matched SNG pair evaluates the supplied bundle, which includes SNG-specific lessons, at one model checkpoint. Repeated runs on held-out tasks with common inputs and budgets could estimate success and cost and separate the contributions of earlier-system and target-specific rules. We plan to measure host resource use and active review time and assess semantic forecasts against later checks.

We plan to extend composition proofs to timing, buffering, and delivery, and add runtime checks for payload correspondence and complete stimulus cycles. Fault injection can exercise the F01 call-recording path. An independent signed-export replay on a frozen engineering case would extend the checkpoint's fixture-based verification tests.

More precise analyzers and stronger proof automation could improve the diagnostics available to the repair loop and discharge additional obligations. We plan to evaluate upgraded tools on frozen artifacts and record their versions, premises, and outcomes before reviewing the affected scope again.

We are investigating NASA's Formal Requirements Elicitation Tool (FRET) to let reviewers examine structured requirements and simulate their interpretation before generation [32]. We plan to verify approval and evidence-handling rules, including scope changes and revocation. The current workbench supports an engineer delegating tasks to agents on a trusted host. A multi-user web service would let engineers delegate scoped reviews and approvals to one another and retrieve the resulting evidence under authenticated roles.

10 Conclusion

SCP Workbench carries selected lessons between isolated workers as versioned rule files and checks each revised target under its own assumptions and approval scope. In SNG, the controller preserved the record and cost of a rejected repair. The authorized continuation links checked component source to a Microkit image and its bounded QEMU observation. This separation of reusable guidance from revision-specific evidence lets engineers reuse experience across targets and review what each resulting system was checked to do.

AI Use

We used OpenAI Codex to propose and repair models, contracts, implementations, and reusable guidance in the engineering workflows. Agents used tool diagnostics to select subsequent proposals. Codex also assisted with inspection of saved records, evidence extracts, figures, drafting, and

Table 7. Inputs and reported outcomes of related work, with the corresponding scope in SCP Workbench.

Work	Starting artifacts and reported outcome	Scope in SCP Workbench
Tahat et al. [31, 45]	GUMBO synthesis and repair in supplied Isolette models, checked by HAMR/Logika over 42 units. Demo supplied Slang logic.	Constructs target models and carries them through code, checks, and execution. Binds approval to the checked revision.
Hardin et al. [19, 21]	HAMR maps architecture and contracts to generated infrastructure, implementation targets, and checks.	Invokes these stages from reviewed requirements and records results against revisions.
AGREE-Dog [44]	AADL contract generation and repair with analyzer feedback.	Retains results and lessons across model, proof, test, and execution stages.
Lee and Heo [29]	Expecto maps natural language to modular test-oracle specifications. Compared against a monolithic LLM and evaluated on Defects4J bugs.	Constructs a system architecture and carries its contracts into language-specific checks and an executable image.
Huang et al. [24]	Natural-language robot tasks to PDDL, SMT-checked plans, and executable behavior trees with reusable skills.	Applies feedback and reuse across architecture, software proofs, and image builds, with recorded scope and approvals.
Zhang et al. [51]	Oxidizer translates eight supplied Go projects to Rust, with type checks and 73% mean function I/O validation using test observations.	Starts from reviewed requirements and checks a generated model and distinct Slang and Rust targets under their stated obligations.
ARc and CLOVER [3, 42]	Policy formalization and symbolic checks in ARc. Documentation, specification, and Dafny code consistency in CLOVER.	Links English and GUMBO to checks on generated Slang and Rust, with recorded assumptions.
AlphaVerus, VeriStruct, VeruSAGE [2, 43, 50]	Verifier-guided Rust/Verus generation, data-structure verification, or proof synthesis.	Uses Verus within a modeling, test, build, and runtime workflow.
WikiSkill and HyperAgents [47, 52]	Retained execution experience supports skill or procedure revision.	Rules carry applicability conditions and links to requirements, checks, and review.
PDD Kit and in-toto [25, 28]	PDD Kit shows intent, proofs, and tests with web-app receipts. In-toto supplies signed artifact statements.	Binds scope and evidence to revisions in an embedded workflow with budget and rule history.

reviewer-style checks of the manuscript. Recorded tool executions supply the reported proof, test, build, and runtime outcomes. Human review governed consequential changes. Appendix B gives the requested model aliases and client versions.

Data-Availability Statement

The manuscript package provides anonymized audit extracts of case results, inventories, log and loader digests, rule histories, costs, and decisions. The claim index is in `evidence/claim-check-index.json`. A separate application checkpoint with pinned dependencies passed the documented setup, 209 tests, and loopback UI and API startup in a clean checkout [7, 8].

We have retained Workbench archives for the 22–23 September 2026 Isolette and historical Producer/Consumer work, the 5 November Producer/Consumer reconstruction, and the 6 November

SNG reconstruction and runtime observation. We plan to submit dated September and November bundles for Artifact Evaluation.[1] An index will identify the evidence and replay scope for each case and link the 8 November approvals to the snapshots they cover. The planned SNG replay includes the frozen model and sources, proof and test inputs, and the recorded loader image. Pinned tools, commands, and expected outputs will support rerunning the checks and booting the image with the recorded decisions held fixed.

References

[1] Alessandro Achille and Stefano Soatto. 2026. AI Agents as Universal Task Solvers. *Entropy* 28, 3 (2026). doi:10.3390/e28030332

[2] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. 2025. AlphaVerus: Bootstrapping Formally Verified Code Generation through Self-Improving Translation and Treefinement. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. 587–615. https://proceedings.mlr.press/v267/aggarwal25a.html

[3] Chenyang An, Sam Bayless, Stefano Buliani, Darion Cassel, Byron Cook, Duncan Clough, Rémi Delmas, Nafi Diallo, Ferhat Erata, Nick Feng, Dimitra Giannakopoulou, Aman Goel, Aditya Gokhale, Joe Hendrix, Victor Heorhiadi, Marc Hudak, Dejan Jovanović, Andrew M. Kent, Benjamin Kiesl-Reiter, Jeffrey J. Kuna, Nadia Labai, Joseph Lilien, Divya Raghunathan, Zvonimir Rakamarić, Niloofar Razavi, Michael Tautschnig, Ali Torkamani, Nathaniel Weir, Michael W. Whalen, and Jianan Yao. 2026. A Neurosymbolic Approach to Natural Language Formalization and Verification. arXiv:2511.09008v2. https://arxiv.org/abs/2511.09008v2

[4] Anonymous. 2026. Producer/Consumer: Experiment comparison and provenance. Author-supplied experiment report. Repository and identifying details withheld for double-blind review.

[5] Anonymous. 2026. SCP: Isolette experiments and development continuations. Author-supplied experiment report. Repository and identifying details withheld for double-blind review.

[6] Anonymous. 2026. SCP Workbench: Recorded Development Results. Repository and identifying details withheld for double-blind review.

[7] Anonymous. 2026. SCP Workbench: Selective Checkpoint Build Verification. Repository and identifying details withheld for double-blind review.

[8] Anonymous. 2026. SCP Workbench: Source Checkpoint Before UI Scaling. Repository and identifying details withheld for double-blind review.

[9] Anonymous. 2026. SNG: Build and the approved 20-second runtime observation. Author-supplied final experiment report. Repository and identifying details withheld for double-blind review.

[10] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to Fix Bugs Automatically. *Proceedings of the ACM on Programming Languages* 3, OOPSLA, Article 159 (2019), 27 pages. doi:10.1145/3360585

[11] Farnaz Behrang, Zhizhou Zhang, Georgian-Vlad Saioc, Peng Liu, and Milind Chabbi. 2025. DR.FIX: Automatically Fixing Data Races at Industry Scale. *Proceedings of the ACM on Programming Languages* 9, PLDI (2025). doi:10.1145/3729265

[12] Jason Belt, John Hatcliff, Robby, John Shackleton, Jim Carciofini, Todd Carpenter, Eric Mercer, Isaac Amundson, Junaid Babar, Darren Cofer, David Hardin, Karl Hoech, Konrad Slind, Ihor Kuz, and Kent Mcleod. 2023. Model-Driven Development for the seL4 Microkernel Using the HAMR Framework. *Journal of Systems Architecture* 134, C (February 2023).

[13] Emery D. Berger, Steve M. Blackburn, Matthias Hauswirth, and Michael Hicks. 2018. SIGPLAN Empirical Evaluation Checklist. https://www.sigplan.org/Resources/EmpiricalEvaluation/. Accessed: 2026-11-05.

[14] Eduardo Cibrián, Jose Olivert-Iserte, Juan Llorens, and Jose María Álvarez-Rodríguez. 2025. An Agent-Based Approach for the Automatic Generation of Valid SysMLv2 Models in Industrial Contexts. *Computers in Industry* 172, Article 104350 (2025). doi:10.1016/j.compind.2025.104350

[15] Collins Aerospace, Loonwerks. [n. d.]. INSPECTA: Industrial Scale Proof Engineering for Critical Trustworthy Applications. https://loonwerks.com/projects/inspecta.html Project overview, accessed Nov. 4, 2026.

[16] Thomas Simon Foster, Bassel Al Omari, Tingchen Fu, Thomas Mann, Carl Domond, Lucia Cipolina-Kun, Bhavul Gauri, Muna Aghamelu, Alexander D. Goldie, Eryk Helenowski, Jean-Christophe Gagnon-Audet, Alberto Pepe, Saba Nazir, Daniel Izcovich, Noam Levi, Rishi Hazra, Karen Hambardzumyan, Nicolas Baldwin, Xian Li, Martin Josifoski, Paris Giampouras, Masoud Jalili Sabet, Anya Sims, Hela Momand, Tatiana Shavrina, Despoina Magka, Jason Weston, Yulin Wang, Anirudh Goyal, João Henriques, Yoram Bachrach, Emily McMilin, and Jakob Nicolaus Foerster. 2026. AI Research Preference Models. arXiv:2608.13940v2. doi:10.48550/arXiv.2608.13940

[1] September archive `[SEPTEMBER-ARTIFACT-URL]`. November archive `[NOVEMBER-ARTIFACT-URL]`.

[17] Wes Gurnee, Nicholas Sofroniew, Adam Pearce, Mateusz Piotrowski, Isaac Kauvar, Runjin Chen, Anna Soligo, Paul Bogdan, Euan Ong, Rowan Wang, Ben Thompson, David Abrahams, Subhash Kantamneni, Emmanuel Ameisen, Joshua Batson, and Jack Lindsey. 2026. Verbalizable Representations Form a Global Workspace in Language Models. arXiv:2607.15495. https://arxiv.org/abs/2607.15495

[18] Stefan Hallerstede, Robby, John Hatcliff, Jason Belt, and David S. Hardin. 2025. Proof Engineering in Logika: Synergistically Integrating Automated and Semi-Automated Program Verification. In *Formal Methods for Industrial Critical Systems (FMICS)*. 39–58. https://loonwerks.com/publications/hallerstede2025fmics.html

[19] David Hardin, Isaac Amundson, Junaid Babar, Darren Cofer, Saqib Hasan, Karl Hoech, Jason Belt, John Hatcliff, Robby, and Stefan Hallerstede. 2025. Automated SysML v2 System Model to Memory-Safe Language Code Generation for Avionics Applications. In *Proceedings of the IEEE/AIAA Digital Avionics Systems Conference (DASC)*. IEEE. doi:10.1109/DASC66011.2025.11257170

[20] John Hatcliff and Jason Belt. 2025. The Isolette System: Illustrating End-to-End Artifacts for Rigorous Model-Based Engineering. In *The Combined Power of Research, Education, and Dissemination*, M. Hinchey and B. Steffen (Eds.). Lecture Notes in Computer Science, Vol. 15240. Springer, Cham, 93–117. doi:10.1007/978-3-031-73887-6_9

[21] John Hatcliff, Jason Belt, Robby, and Todd Carpenter. 2021. HAMR: An AADL Multi-Platform Code Generation Toolset. In *10th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA) (LNCS, Vol. 13036)*. 274–295.

[22] John Hatcliff, Danielle Stewart, Jason Belt, Robby, and August Schwerdfeger. 2023. An AADL Contract Language Supporting Integrated Model- and Code-Level Verification. *Ada Letters* 42, 2 (April 2023), 45–54. doi:10.1145/3591335.3591339

[23] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. 2022. Meta-Learning in Neural Networks: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 9 (2022), 5149–5169. doi:10.1109/TPAMI.2021.3079209

[24] Yu Huang, Ziji Wu, Zhengyi Ma, Kexin Ma, and Ji Wang. 2026. Neuro-symbolic Hierarchical Learning for Long-Horizon Robotic Tasks. *Proceedings of the ACM on Programming Languages* 10, PLDI (2026). doi:10.1145/3808338

[25] in-toto Project. [n. d.]. Attestation Framework: Statement, Version 1. https://github.com/in-toto/attestation/blob/main/spec/v1/statement.md

[26] Saketh Ram Kasibatla, Arpan Agrawal, Yuriy Brun, Sorin Lerner, Talia Ringer, and Emily First. 2026. Cobblestone: A Divide-and-Conquer Approach for Automating Formal Verification. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. doi:10.1145/3744916.3773178

[27] Hyunsu Kim, Mukund Raghothaman, and Kihong Heo. 2022. Learning Probabilistic Models for Static Analysis Alarms. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. 1282–1293. doi:10.1145/3510003.3510098

[28] Alex Lavaee and Matthai Philipose. 2026. Formally Verifying Agent-Written Full-Stack Web Apps with PDD Kit. Microsoft Research Blog. https://www.microsoft.com/en-us/research/blog/formally-verifying-agent-written-full-stack-web-apps-with-pdd-kit/ Research prototype, published November 5.

[29] Dongjae Lee and Kihong Heo. 2026. Expecto: Extracting Formal Specifications from Natural Language Description for Trustworthy Oracles. *Proceedings of the ACM on Programming Languages* 10, PLDI (2026). doi:10.1145/3808332

[30] David L. Lempia and Steven P. Miller. 2009. *Requirements Engineering Management Handbook*. Federal Aviation Administration. https://www.faa.gov/sites/faa.gov/files/aircraft/air_cert/design_approvals/air_software/AR-08-32.pdf DOT/FAA/AR-08/32.

[31] Loonwerks. 2026. INSPECTA Spec-Code-Proof Copilot. GitHub repository. https://github.com/loonwerks/INSPECTA-Spec-Code-Proof-Copilot/tree/main?tab=readme-ov-file Accessed: 2026-05-06.

[32] NASA. [n. d.]. FRET: Formal Requirements Elicitation Tool. NASA Software Catalog, ARC-18066-1. https://software.nasa.gov/software/ARC-18066-1 Accessed: 8 November 2026.

[33] National Institute of Standards and Technology. 2015. *Secure Hash Standard (SHS)*. Technical Report FIPS PUB 180-4. U.S. Department of Commerce. doi:10.6028/NIST.FIPS.180-4

[34] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv:2506.13131. doi:10.48550/arXiv.2506.13131

[35] Object Management Group. 2025. OMG System Modeling Language (SysML) v2 Specification. https://www.omg.org/spec/SYSML

[36] OpenAI. 2026. GPT-6 Astra Model and API Pricing. https://developers.openai.com/api/docs/models/gpt-6-astra. Accessed November 8, 2026.

[37] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating Agent-Based Program Repair at Google. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 365–376. doi:10.1109/ICSE-SEIP66354.2025.00038

[38] SAnToS Laboratory. 2026. HAMR Tutorials: SysML v2 and Rust Examples. https://github.com/santoslab/hamr-tutorials/tree/01e7558aec35272052a8c522bc35618d5dc1f6f4 Repository revision 01e7558aec3527; accessed 6 November 2026.

[39] Secure Systems Lab. [n. d.]. Dead Simple Signing Envelope (DSSE): Protocol. https://github.com/secure-systems-lab/dsse/blob/master/protocol.md

[40] seL4 Foundation. 2026. *Microkit User Manual* (2.4.0 ed.). https://docs.sel4.systems/projects/microkit/manual/latest/ Accessed November 8, 2026.

[41] Sireum HAMR Project. [n. d.]. How GUMBO Contracts are Translated to Rust Boolean Functions to Provide GUMBOX Executable Contracts. https://hamr.sireum.org/hamr-doc/gumbox-translation/ Accessed Nov. 5, 2026.

[42] Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. 2024. Clover: Closed-Loop Verifiable Code Generation. In *Proceedings of the First International Symposium on AI Verification (SAIV)*. Springer, 134–155. doi:10.1007/978-3-031-65112-0_7

[43] Chuyue Sun, Yican Sun, Daneshvar Amrollahi, Ethan Zhang, Shuvendu Lahiri, Shan Lu, David Dill, and Clark Barrett. 2025. VeriStruct: AI-assisted Automated Verification of Data-Structure Modules in Verus. arXiv:2510.25015 [cs.SE] https://arxiv.org/abs/2510.25015

[44] Amer Tahat, Isaac Amundson, David Hardin, and Darren Cofer. 2025. AGREE-Dog Copilot: A Neuro-Symbolic Approach to Enhanced Model-Based Systems Engineering. In *Bridging the Gap Between AI and Reality (AISoLA 2025), Selected Papers*. doi:10.1007/978-3-032-07132-3_8

[45] Amer Tahat, David Hardin, Isaac Amundson, Junaid Babar, Timothy A. Barclay, Janet Liu, Karl Hoech, and Darren Cofer. 2026. Bridging Natural Language to Verified Implementation: A Neuro-Symbolic High-Assurance MBSE Pipeline in SysML v2. In *Proceedings of the Digital Avionics Systems Conference (DASC)*. https://loonwerks.com/publications/tahat2026dasc.html

[46] Amer Tahat, David Hardin, Isaac Amundson, Junaid Babar, Timothy A. Barclay, Janet Liu, Karl Hoech, and Darren Cofer. 2026. Bridging Natural Language to Verified Implementation: A Neuro-Symbolic High-Assurance MBSE Pipeline in SysML v2. DASC conference presentation. September 17, 2026, slides 4, 7, 8, 9, 11, 12, and 18.

[47] Liyan Tang, Cyrus Rashtchian, Chun-Sung Ferng, Andrew Tomkins, Da-Cheng Juan, and Tu Vu. 2026. WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution. arXiv:2608.27454. https://arxiv.org/abs/2608.27454

[48] Verus Project. [n. d.]. Verus Tutorial and Reference: Overview. https://verus-lang.github.io/verus/guide/overview.html.

[49] Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. 2023. Transformers Learn In-Context by Gradient Descent. In *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 35151–35174.

[50] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. 2026. VeruSAGE: A Study of Agent-Based Verification for Rust Systems. arXiv:2512.18436v2. https://arxiv.org/abs/2512.18436v2

[51] Hanliang Zhang, Cristina David, Meng Wang, Brandon Paulsen, and Daniel Kroening. 2025. Scalable, Validated Code Translation of Entire Projects using Large Language Models. *Proceedings of the ACM on Programming Languages* 9, PLDI, Article 212 (2025), 25 pages. doi:10.1145/3729315

[52] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. 2026. HyperAgents. arXiv:2603.19461v1. https://arxiv.org/abs/2603.19461v1

A Supporting Evaluation Details

The case reports document the inputs and adaptations [4, 5, 9] and use the HAMR workflow [20, 38]. This appendix records learning, costs, and review dates for Section 7. The extracts `evidence/sng-learning-records.json` and `evidence/evidence-audit.json` retain lesson texts, results, and hashes.

Inputs. The later SNG reconstruction used Codex 0.155.1 and requested `gpt-6-astra`. The tools were Sireum `4.20260810.80aad0c2` and cvc5 1.3.4, revision `f3b21c4`. Both arms received `common_inputs/requirements.md`, the same libraries and guides, and the approved choices in `common_inputs/design_supplement.json`. The protocol and attempt contexts identify these inputs. Appendix B records the worker configuration.

Additional Isolette checks. The separate composition probe verified five conditions and reported two verification errors. A separate assisted Isolette candidate passed parsing/types and architecture capture, then failed its directed-graph comparison [5, 6]. The completed Rust continuation in Section 7 has its own candidate and evidence records.

Producer/Consumer input and accounting boundaries. The first construction supplied English requirements and existing rulebooks for a new target. The reported dispatch inputs come from the research team's account. The checkpoint workflow guide independently documents Producer/Consumer as held out of the Monitor training split [8]. The original first-dispatch manifest would identify the exact file bytes. Later, the paired reconstruction's guided arm added six general entries and three rules from target-specific repairs. Table 16 attributes 257,355 measured tokens across historical generation and English preparation. Its elapsed span covers those attributed calls and gaps.

First SNG construction and subsequent learning. The first SNG K^+ dispatch contained eleven retained entries from general rules and earlier systems, including Producer/Consumer. Both first-run arms received common guides with SNG model, implementation, and test examples. The guided arm also received approved reference-derived design choices. The first guided candidate passed its in-loop model checkpoint at 397,520 measured tokens after five repairs, with two checked connection units and full English review pending. The ordinary-input baseline stopped at 387,144 tokens after six repairs. The earlier run summary is preserved in `evidence/timing-model-audit.json`.

Seven draft observations in `LEARNING_OBSERVATIONS_DRAFT.json` and four in `LEARNING_OBSERVATIONS_CONTINUATION.json` arose from the early SNG attempts and recoveries. All eleven texts occur among the 66 entries supplied to the later shared-supplement reconstruction. These entries were selected for the experiment, with qualification for publication still open. For example, a parse failure produced guidance to use SysML literals such as `0 [s32]` in a model. The rendered Slang notation belongs in the generated code. The later candidate received the lesson before passing its model checks after repair. The later comparison tests the full guidance bundle.

Cumulative cost and completion. Table 8 shows successive SNG K^+ milestones. Each increment contributes to the cumulative experiment-model cost of the same task. The rejected proposal removed an existing test function; its usage remains charged. Later authorization retained the candidate and earlier consumption.

The later K^+ trajectory used 346,224 + 78,469 + 151,909 = 576,602 measured experiment-model tokens through observation. The later K arm spent 388,726 tokens at its model-stage repair limit. Its dependent checks retain `NOT_RUN` status. Appendix B reports historical acquisition and supporting sessions separately.

Table 8. SNG K^+ experiment-model tokens through successive milestones.

Recorded point	Increment	Cumulative
Five model checks passed	346,224	346,224
Implementation proposal rejected	78,469	424,693
Authorized paid repairs	151,909	576,602
Later deterministic repairs and runtime	0	576,602

Requirements review and release dates. Table 9 orders the recorded input approvals, engineering results, source reviews, and release decisions. It shows when each review entered the evidence chain.

Table 9. Recorded review chronology, in UTC. Times identify the stated event or decision record.

Case	Date and time	Event
Isolette	23 Sep, 03:30:27	Revised input approved.
	23 Sep, 03:52:54	Revised generation completed.
Producer/Consumer reconstruction	5 Nov, 16:33:50	K^+ model run completed.
	5 Nov, 16:34:01	Criteria recorded before evaluator inspection.
	5 Nov, 16:43:56	Source audit recorded.
Later SNG	6 Nov, 07:32:15	Shared-supplement protocol recorded nine seconds after this reconstruction's first K dispatch, before inspecting new candidates.
	6 Nov, 08:12:36	Terminal source review recorded.
	6 Nov, 10:20:59	Runtime observation ended.
	6 Nov, 10:52:13	Broader requirements audit recorded.
	6 Nov, 10:54:22	Named-contract audit recorded.
All three	8 Nov, 18:17:43	Owner release approvals recorded.

Isolette source review used FAA requirements, transcribed guards and timing constraints, and repaired component evidence. Producer/Consumer's audit classified 38 source-checklist items: 36 model/scope items and two workflow constraints. The K^+ review marked 34 represented and four satisfied. SNG's named-contract audit linked six English component relations to contracts and recorded proofs under stated premises. Appendix E follows these requirement links.

Reviewer `POWERUSERS-1`'s release receipts record approval of the three case reviews. Isolette's included scope covers timeout-priority component verification, tests, build, and observation under the environmental assumptions and physical-operator exclusion. Producer/Consumer's covers the paired K^+ model, directed architecture comparison, and source-conformance review. SNG's covers the completed model and six downstream categories for its initialization-repair-8 source and approved observation. The receipts identify candidate and evidence hashes and retain the remaining system obligations. The extract `evidence/accounting-and-release.json` preserves these dated decisions; changed contents, assumptions, evidence, or scope require renewed review.

Checks of retained evidence. A read-only audit matched the same 201 paths in SNG's build and observation inventories. These comprise 108 Microkit workspace files and 93 Slang workspace files, including 89 `.scala`, 58 `.rs`, 16 `.c`, and 10 `.h` paths plus 28 other project files. The audit also matched four completion links, four log digests, and the loader digest. Subtracting readiness from observation end reproduced 20.033 seconds. The runtime log contained 18 receiver messages. The

Isolette release receipt identifies 217 implementation files. The later Producer/Consumer release identifies 16 model-check inputs. Each count describes its own inventory.

Source-line inventory. The 8 November source audit identifies the measured snapshots and retains per-file counts and fingerprints. The anonymized summary in `evidence/source-line-counts.json` records the original aggregate counts. The follow-up role audit in Appendix A.1 adds file counts and includes the model associated with the retained Isolette Scala project.

Workbench Python footprint. The local Workbench inventory reports 8,255 implementation lines in 51 Python files, 3,415 automated-test lines in 44 files, and 2,097 validation and support lines in 35 files, totaling 13,767 lines in 130 files. The summary reports agreement between two counting methods. The anonymized counts in `evidence/workbench-python-inventory.json` describe the Workbench snapshot. Table 3 describes the system builds.

Compact evidence for inspection and replay. The current extracts support inspection of the reported provenance and outcomes. For independent replay, the case can be distributed at its checked endpoint. Table 10 identifies the files needed for each purpose. A local setup recipe can recreate the tool environment. If a receipt is redacted for release, the manifest can record its original fingerprint separately from the fingerprint of the released excerpt.

Table 10. Records for independently inspecting the historical run and repeating the checked SNG endpoint.

Question	Smallest relevant records or artifacts
What entered the worker?	Initial dispatch manifest with file roles and fingerprints, task inputs, selected rule versions and their origins. Record common inputs and the selected bundle separately for the paired runs.
What did the engineer decide?	Decisive diagnostics and proposal outcomes, approval records with assumptions and scope, and links to the affected revisions.
What did the tools check?	Frozen model, source, proof and test inputs, the build inventory, selected check logs and receipts, and pinned tool versions and commands.
What image ran?	The 4,957,232-byte loader image and its fingerprint, QEMU launch command and observation configuration, and the matching receipt and output. The recorded observation contains 18 receiver messages over 20.033 s after readiness.

Selected tool records. Table 11 summarizes the archived evidence visible in the submitted audit extracts. A tool result identifies what was checked. The artifact and receipt fingerprints allow the reported result to be associated with the version that was checked.

The controller-test extract in `evidence/evidence-audit.json` records the 45 expected/actual assertions and the F01/control traces summarized in Table 6.

A.1 Source Lines by File Role

The authors' local source audit classifies each file once by its primary purpose, including test-related files under `src/main`. Tables 12, 13, and 14 report aggregate counts as lines (files). Lines are nonblank physical source lines after removal of ordinary comments. The classification counts whole files. For example, serialization, interface, and conditional-test files may contain lines serving more than one purpose. The model rows include GUMBO contracts, and the implementation roles can include embedded contracts and proofs. The local audit tabulates shared SysML libraries and

Table 11. Tool evidence summarized from the case audit extracts. The rows report different checks and may refer to different candidates.

Stage	Archived record excerpt
Sireum and HAMR	Later SNG guided model passed parsing and typing, Rust/Microkit and Slang/JVM generation, and architecture capture. An earlier repair record preserves the SysML signed-literal diagnostic and retained guidance.
Logika with cvc5	The later Producer/Consumer model record passes the connection implication from producer range [0,90] to consumer assumption [0,100]. The SNG configuration names cvc5 1.3.4 and the SNG model checkpoint records a passing Logika integration check.
Verus	The SNG continuation reports 32 target-crate units verified with zero errors, including six initialization and composition units. The extract records the Verus outcome.
GumboX	SNG reports 33 Rust test functions and 808 Slang/JVM test cases passing. The two branches use different counting units.
Microkit and QEMU	Build receipt records exit code 0 and matches the loader digest. The observation receipt and log record 20.033 s after readiness and 18 receiver messages.

C/header support separately. The authors retain the per-file classification CSV locally. This appendix provides the aggregate counts.

Table 12. SysML v2 and GUMBO source by recorded model snapshot.

Model snapshot	Lines (files)
Isolette successful Rust/Microkit build	936 (8)
Producer/Consumer Rust/Microkit resumption	73 (1)
Producer/Consumer paired K^+ JVM generation	73 (1)
SNG model shared by both backends	202 (5)
Isolette current retained model	846 (8)

Table 13. Rust/Verus source by file role in the three successful build inventories. P/C denotes Producer/Consumer.

Role	Isolette build	P/C resumption	SNG build
Component implementation and embedded contracts	833 (11)	106 (2)	291 (2)
Runtime sender/receiver components	0	0	133 (2)
Port APIs and foreign-interface code	2,683 (22)	197 (4)	478 (8)
Module, logging, and runtime support	1,067 (44)	194 (8)	386 (16)
Executable contract checkers	1,682 (11)	120 (2)	454 (2)
Data types and supporting definitions	188 (15)	35 (5)	53 (6)
Dedicated tests and test support	4,974 (66)	537 (12)	944 (22)
Total	**11,427 (169)**	**1,189 (33)**	**2,739 (58)**

Table 14. Slang/Scala source by file role in the retained Isolette project, the Producer/Consumer guided JVM checkpoint, and the SNG build.

Role	Isolette retained	P/C paired K^+	SNG build
Component entrypoints and embedded contracts	777 (15)	37 (2)	92 (2)
Environment simulation, runtime observation, and monitoring	2,741 (35)	0	53 (2)
Architecture, port APIs, and bridges	2,775 (27)	502 (9)	826 (13)
Contract checkers and state containers	1,107 (20)	103 (5)	205 (5)
Serialization and data utilities	7,806 (16)	2,458 (5)	2,589 (6)
Test cases, harnesses, and support	5,394 (106)	864 (20)	986 (26)
Test-data generators	8,777 (15)	3,522 (6)	3,645 (6)
Bundled ART runtime library	1,796 (28)	1,796 (28)	1,796 (28)
Total	**31,173 (262)**	**9,282 (75)**	**10,192 (88)**

The retained Isolette model (846 lines) and Slang/Scala project (31,173 lines) total 32,019 lines in 270 files. The successful Rust build has its own 936-line model and 11,427 Rust/Verus lines, totaling 12,363 lines.

SNG has 1,930 lines of dedicated tests, harnesses, and support across both backends. Its test-data generators contribute another 3,645 lines. Together these roles account for 5,575 of 13,133 lines (42.5%) in the counted build inventory. Table 2 reports executed tests in their own units, with 33 Rust test functions and 808 Slang/JVM test cases. The Slang/Scala role inventory classifies 88 SNG files. The matched build and observation inventories record 89 `.scala` paths within their 201 paths. The two counts refer to their respective inventories.

B Recorded Time, Configuration, and Additional Costs

This appendix records worker usage, historical preparation, and supporting sessions. Call receipts, controller records, and session logs supply the values. The extracts `evidence/timing-model-audit.json`, `evidence/support-session-usage.json`, and `evidence/accounting-and-release.json` retain the inputs, intervals, hashes, and release decisions.

Recorded configuration. The 117 receipts record requests for `gpt-6-astra`. They identify the same Codex client binary, version 0.155.1. The SNG worker matches its frozen source hash. Its options include `--strict-config`, `--ignore-user-config`, `--ignore-rules`, and `--ephemeral`. It uses a temporary configuration directory and a JSON output schema. Appendix A identifies the Sireum and cvc5 versions.

Intervals in the main evaluation. Table 15 gives dispatch, check, and elapsed times. On 6 November 2026, SNG K^+ started at 07:49:38.630693 UTC. Observation ended at 10:20:59.935633 UTC. The difference is 9,081.305 seconds, or 2 h 31 min 21.3 s. It spans the initial model phase, rejected proposal, authorized repairs, approval pauses, and final observation. Model dispatch across the eight experiment calls totals 1,123.279 seconds (18.72 minutes). The final observation lasts 20.033 seconds after readiness.

Producer/Consumer's separate, unpaired baseline follow-up used 305,087 measured tokens, with 358.59 seconds of model dispatch and checks across a 940.49-second elapsed interval. The interval includes 567.48 seconds of harness recovery and authorization. Each of the two interrupted Producer/Consumer runs retains a separate 1M estimated usage debit [4].

Table 15. Measured experiment-model tokens and time in seconds. Dispatch includes call handling. Elapsed time includes interruptions and pauses. SNG milestones are cumulative except for the paid-repair increment. Dashes mark unavailable totals.

Recorded point	Tokens	Dispatch	Checks	Elapsed
Later Producer/Consumer K^+	179,515	109.53	94.45	210.19
Later Producer/Consumer K, interrupted	48,236	36.35	17.40	57.86
Later SNG K, repair limit	388,726	682.71	362.30	1,052.77
Later SNG K^+, model checks	346,224	532.43	154.10	—
SNG K^+, proposal rejected	424,693	785.67	154.10	949.80
SNG K^+, paid repair increment	151,909	337.61	280.18	627.67
SNG K^+, through observation	576,602	1,123.28	—	9,081.30

Experiment-worker ledger. Table 5 in the main evaluation gives the disjoint case totals through SNG closeout. The parent campaign's 67 calls used 3,060,221 measured tokens plus 2M estimated debits under its 6,310,229 ceiling. Fifty earlier calls contribute the remaining 6,256,540 accounted tokens.

A ledger observation at 18:17:25 UTC on 8 November includes 24 later BASS calls, adding 1,382,740 measured tokens. The broader balance then reached 12,699,501 accounted tokens, with a parent ceiling of 7,560,221. Successive BASS reports contain overlapping sets of 14, 19, and 24 calls; the later balance counts the final 24-call set once. The three-case accounting in Table 5 uses the earlier SNG closeout.

Historical knowledge construction. Table 16 gives acquisition and preparation costs from the earlier ledger [4, 5]. Call minutes sum intervals from reservation to usage recording. Elapsed minutes span the first attributed reservation through the last usage record, including gaps. The case spans overlap in calendar time.

Table 16. Historical acquisition and preparation, 22–23 September 2026. Table 5 includes these costs. Call minutes sum reservation-to-usage intervals; elapsed minutes cover the timestamp span.

Attribution	Measured tokens	Est. debit	Call min	Elapsed min
Isolette	341,178	1M	32.43	247.22
Producer/Consumer	257,355	0	7.66	137.03
Shared preparation	658,007	4M	43.30	202.82

Isolette's 341,178 tokens comprise 315,342 for generation and 25,836 for English preparation. Producer/Consumer's 257,355 comprise 242,186 and 15,169, respectively; its English-preparation call followed the earlier reconstruction calls. Shared preparation comprises 529,698 tokens for rule learning, 101,134 for Monitor preparation, 15,277 for unit definitions, and 11,898 for a probe. The ledger retains shared preparation under its own attribution.

Supporting sessions. Table 17 reports three SNG sessions for context preparation, authorization checks, and proof assistance. Their saved contexts request `gpt-6-astra` with reasoning effort `ultra` through Codex 0.167.0. The sessions began between 07:08:30 and 07:09:10 UTC on 6 November, and the table uses each session's last cumulative counter before the 10:20:59 observation end. This period covers preparation, both comparison arms, and continuation. The counters increase monotonically, and each first counter equals its first call's usage.

Table 17. SNG supporting-session tokens through the observation cutoff. Cached and other input are disjoint parts of input; output includes reasoning tokens. Each row uses one final cumulative counter.

Session role	Cached input	Other input	Output
Context preparation	46,523,776	426,670	170,060
Authorization checks	40,697,216	320,279	171,754
Proof assistance	42,618,880	539,721	137,373
Selected total	129,839,872	1,286,670	479,187

These three sessions total 131,605,729 tokens, including repeated cached context. The 28M allowance applies to experiment workers. The supporting counters cover preparation, both arms, and continuation.

Illustrative API repricing. For request r, let I_r be total input, H_r the cached input within it, and O_r output, including recorded reasoning tokens. With rates p_u, p_h, p_o for uncached input, cached input, and output in USD per million tokens, the illustrative model-token cost is

$$\widehat{C}_{\mathrm{API}} = \frac{1}{10^6} \sum_r \left[p_u(r)(I_r - H_r) + p_h(r)H_r + p_o(r)O_r \right].$$

GPT-6 Astra Standard rates are $(10, 1, 50)$ USD per million for up to 272,000 input tokens. Longer requests use $(20, 2, 75)$ [36]. The estimate prices the recorded token classes. Cache-write charges, estimated debits, tool and host costs, and labor are separate.

Table 18. Illustrative base-rate prices for the later SNG comparison and supporting sessions. The guided checkpoint is included in the guided observation row.

Recorded scope	Uncached input	Cached input	Output	USD
K, model-stage stop	352,589	11,776	24,361	$4.76
K^+, model checkpoint	326,487	0	19,737	$4.25
K^+, through observation	529,778	0	46,824	$7.64
Three supporting sessions	1,286,670	129,839,872	479,187	$166.67

The 15 selected worker calls had at most 70,503 input tokens per request. The supporting extract provides cumulative counters with zero recorded cache writes. At base rates its 129,839,872 cached input tokens contribute $129.84 of the $166.67 estimate. The counters leave the per-request tier mix unknown. Pricing all supporting tokens at long-prompt rates gives an alternate scenario of $321.35.

Model cost per source line. Let L be the source lines in the delivered build (Table 3). For the guided SNG worker through observation, $L = 13{,}133$. The calculation gives

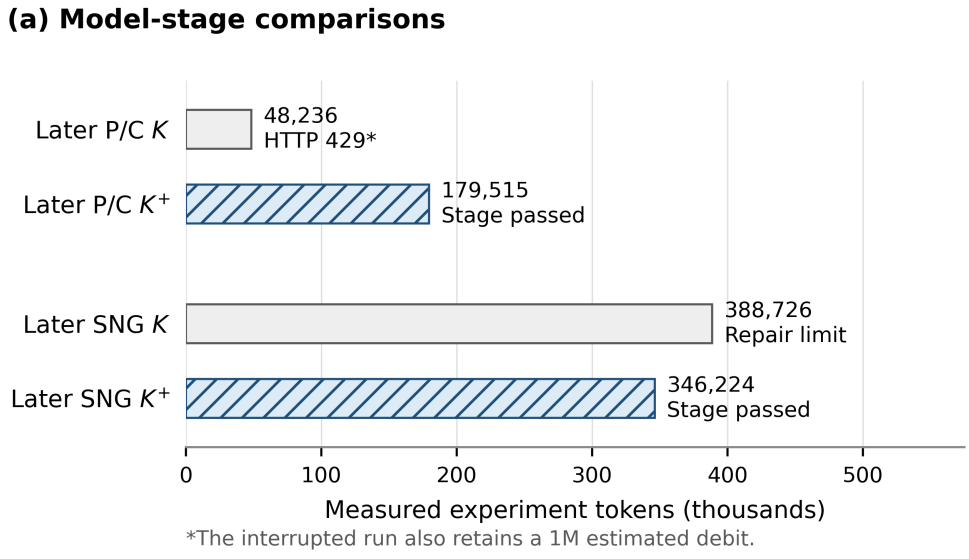
$$\frac{10(529{,}778) + 1(0) + 50(46{,}824)}{10^6} = \$7.63898, \qquad \frac{\$7.63898}{13{,}133} \approx \$0.000582 \text{ per line.}$$

This is $0.58 per 1,000 source lines. Charging the stopped baseline, guided worker, and all three supporting sessions to the one delivered build would give $179.06, or $13.63 per 1,000 lines, in the base-rate scenario. The source-line denominator measures artifact size. Table 2 reports the formal obligations, tests, build, and observation separately.

Tokens per completed target. For T measured tokens and v completed targets, we display T/v when $v > 0$, and $+\infty$ as a convention when $v = 0$. The later SNG model target requires all five declared checks. Each arm had a 500,000-token allowance. The baseline spent 388,726 at its repair limit and completed 0/1 targets. Its eventual cost to reach completion is unknown. The guided arm completed 1/1 at 346,224 tokens. The baseline's dependent checks have `NOT_RUN` status.

Tahat et al. [45] used illustrative rates of $(60, 0, 270)$ USD per million for long-context uncached input, cached input, and output, respectively, and normalized cost by verified GUMBO units. They reported about 9.87M context tokens per verified unit at 2/42 and 329k at 42/42 with one-time rule preparation included [45, 46]. The SNG source-line calculation uses the Astra rate card and the delivered build's size. Estimating how unit cost changes with campaign size requires measurements at multiple sizes.

Figure 6 gives the detailed paid milestones. The inputs and derivations for the figure and the price table are in `evidence/comparison-plot-data.json` and `evidence/cost-normalizatio n.json`.



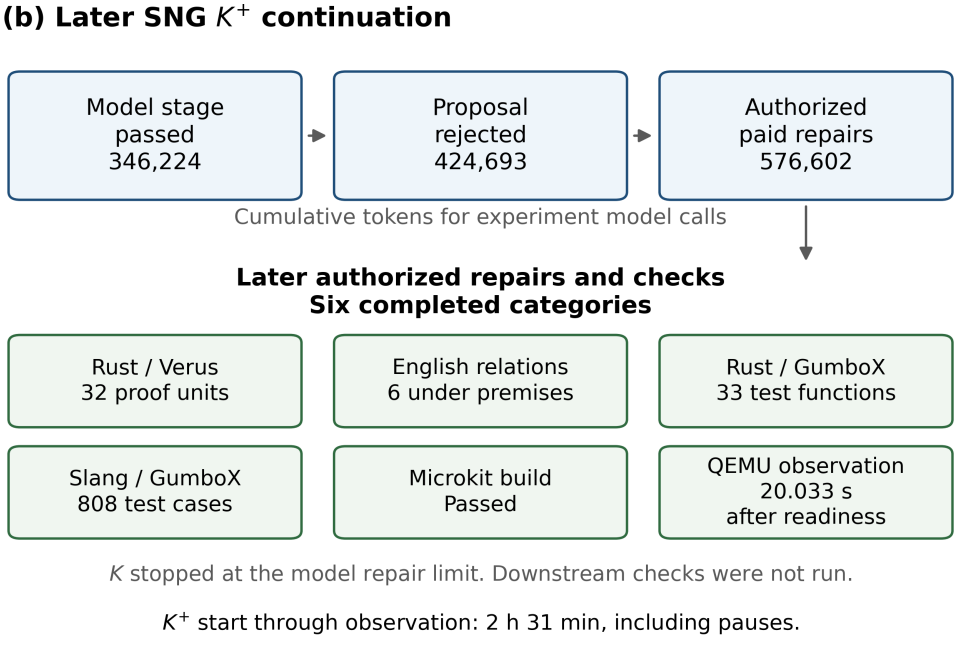


Fig. 6. Later model comparisons for Producer/Consumer (P/C) and SNG. Panel (a) shows measured tokens through each model outcome. Panel (b) follows the later SNG K^+ candidate through the rejected proposal, paid repairs, and final checks. Serializer and initialization repairs after the paid calls add no measured experiment-model tokens. Appendix A reports the first SNG construction separately.

C Requirements Review Procedure and Case Evidence

Algorithm 3 details Figure 7. Campaign record C retains the English inputs, requirement table, candidates, costs, evidence, and decisions. Task T names the reviewed scope, checks, and limits; R identifies selected rule revisions. The table maps each source obligation to its interpretation, representation, assumptions, evidence, and review status.

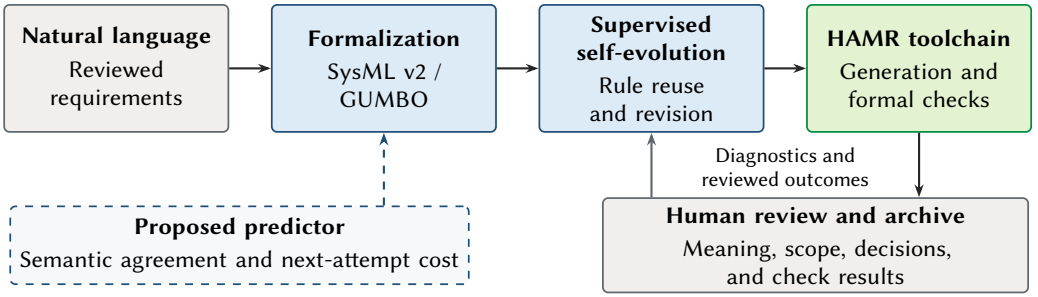


Fig. 7. From reviewed natural-language requirements to checked artifacts, inspired by ARc's formalization and feedback loop [3, Fig. 1, p. 4]. Diagnostics and reviewed outcomes inform repairs and rule updates. The dashed predictor would advise which candidate to check next. Actual allowance and recorded approvals govern attempts.

This procedure composes development, accounting, and review services [8]. The reported Producer/Consumer and SNG cases used saved source and proof audits. Their review criteria followed the start of generation; SNG's final six-relation audit followed runtime. Appendix E gives the actual mappings and service configuration.

Appendix D describes the proposed use of predictions to select attempts. Historical engineering results retain their status; later owner release decisions record their dates and scopes.

Algorithm 3. Review requirements fidelity during development

Input: Campaign record C; task T; selected rule revisions R.
Output: Updated C and scoped task outcome q.

Retain the original English, input versions, approved scope, and rule revisions
Read the source line by line; record obligations, interpretations, representations, and assumptions in the requirement table
Assess meaning and unresolved questions; record the responsible reviewer or agent
if T lacks current human approval **then return** PENDING
while authority, deadline, and allowance after consumption and reservations permit an attempt **do**
    if usage is unresolved **then return** PENDING
    if reviewed inputs or dependencies have changed **then return** STALE
    Reserve allowance; request a proposal for T using R
    Record usage, proposal, and detailed call outcome
    if response is invalid or check time is unavailable **then return** PENDING
    Run HAMR and the required checks; retain results, failures, obligation links, and content fingerprints
    Update the requirement table; review interpretation, assumptions, and unresolved obligations
    Apply current human decisions with Algorithm 1; request any required new review; obtain q
    if q is accepted **then**
        Mark the included task scope complete; record the owner's release decision when supplied, with exclusions and remaining obligations
        return q
    if q is stale or rejected **then return** q
    if the next action changes meaning, scope, assumptions, or limits **then**
        Retain the proposal; **return** PENDING for human review
    if rule revision follows a current failed `NOT_VALIDATED` use of the same rule and learning plan **then return** `NOT_VALIDATED`
    if the next action lacks repair authority **then return** q
    Retain earlier charges and failures; refine within the approved scope and retry
return PENDING for further allowance or instruction

D Early Assessment and Budgeted Search

An early semantic assessment asks whether a proposed interpretation is likely to preserve the source obligations and assumptions. Its prediction can guide which candidate receives the next formalization attempt. Research Preference Models motivate assessing candidates before costly execution [16]; reusable algorithmic structure provides a reason to expect earlier experience to help the search [1]. Verification and human review then assess the resulting artifacts and their scope.

D.1 Semantic Prediction and Its Geometry

The workbench supplies the comparisons and forecasts in Table 19 [8]. Automatic use of their predictions to admit attempts and rank search branches is a proposed extension. The later SNG reconstruction supplied 66 preselected entries in its K^+ input and used that set throughout. Section 6 describes the separate source and contract reviews.

Table 19. Comparison inputs and outputs in the inspected implementation.

Mechanism	Inputs	Output and use
Rule retrieval	Task query; rule trigger and principle	Lexical cosine distance; selects related guidance.
Contract comparison	Features of assumptions and guarantees after architecture assessment	Cosine distance; supports review of contract differences.
Model forecast	Reviewed learning material, declared task, and available feedback	Model belief score and explanation for the next application.

For rule retrieval, x counts words and symbols in the task query and y those in a rule's trigger and principle. Nonzero vectors satisfy the configured comparison when

$$d_{\cos}(x, y) = 1 - \frac{x \cdot y}{\|x\|\|y\|} \leq \epsilon. \tag{1}$$

The retrieval default is $\epsilon = 0.1$, with allowed values in $[0, 0.3]$. Contract comparison uses features of assumptions and guarantees after architecture checking. These representations approximate similarity: counting words and symbols gives $a \leq b$ and $b \leq a$ the same representation. A model assessment can examine the relation and explain uncertainty. A distance limit corresponds to an angle $\theta_{\max} = \arccos(1 - \epsilon_{\max})$ around the reference direction.

Figure 8 retains the reference, candidate directions, and applicability boundary illustrated by Tahat et al. [46, slide 12]. Green and check marks denote a favorable prediction that sends a candidate to formal assurance within the remaining budget. Crosses identify illustrative candidates for refinement or review. The construction extends to any specification language for which paired requirements, candidate representations, and suitable checks are defined; the engineering cases here use SysML/GUMBO. Forecast calibration is a subject for future evaluation.

D.2 Rule Application and Budgeted Search

Changing ϵ broadens or narrows the similarity comparison. Revising a rule changes its conditions and procedure. Each application retains the selected version and supporting evidence.

Figure 10 follows Tahat et al. [46, slide 7] from requirements through formalization and placement to code and proof. An incorrect assumption or placement can affect several downstream artifacts. Checking an earlier choice can reveal a repair before those descendants are generated. The displayed branch counts illustrate this dependency.

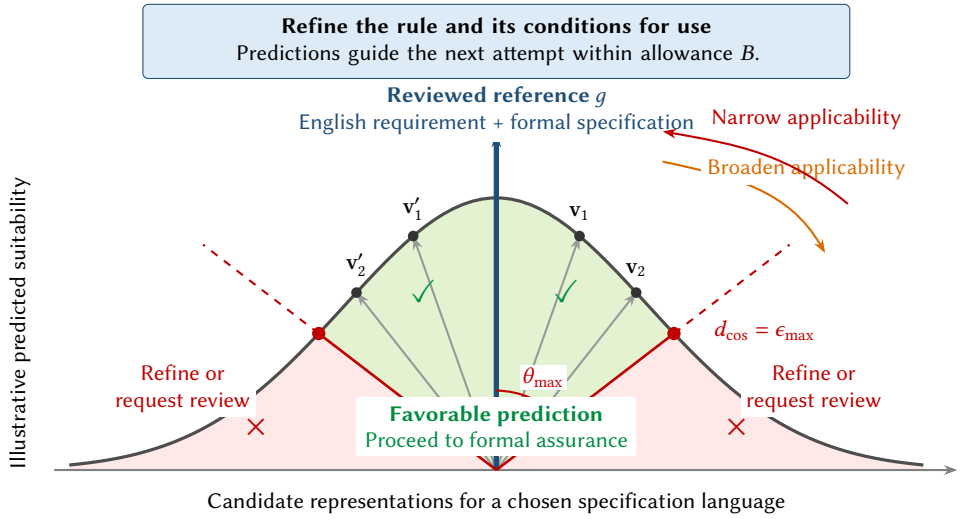


Fig. 8. Semantic prediction around a reviewed reference, redrawn from Tahat et al. [46, slide 12]. Green and check marks indicate favorable predictions: proceed to formal assurance. Red and crosses indicate refinement or review. Positions, heights, and marks are illustrative. The geometry applies to a chosen representation in any specification language; formal checks and human decisions establish the accepted scope.

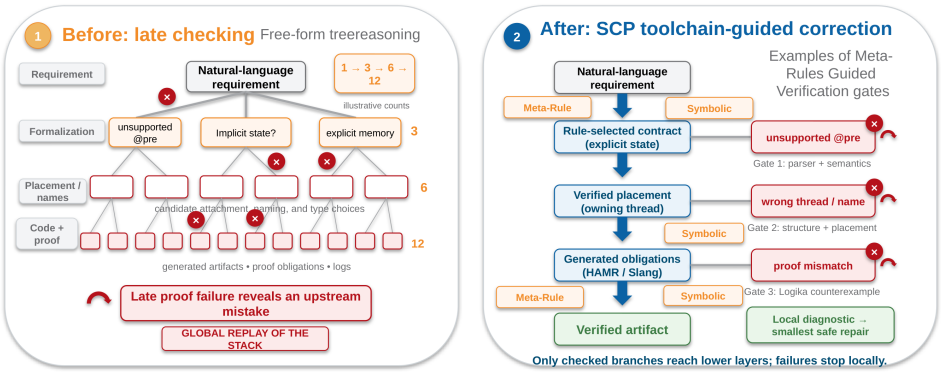


Fig. 9. Late checking and local repair, cropped from the presentation by Tahat et al. [46]. The left panel illustrates how an early choice can multiply downstream artifacts before a failure is detected. The right panel shows checks on formalization, placement, and proof that return diagnostics at the failing stage. The branch counts are illustrative.

Proposed agreement and cost prediction. For a requirement and candidate under stated premises, an agreement model could learn from human review outcomes, using the on-demand forecast and contract-feature similarity as inputs. A cost model could estimate incremental wall time and model tokens from attempts with comparable artifact sizes, check plans, and tool versions. Candidate ordering would favor likely agreement within the remaining time and token allowance. Estimates with sparse supporting records would carry uncertainty and could trigger review. Linking predictions to observed costs and review outcomes would let us evaluate calibration and saved effort.

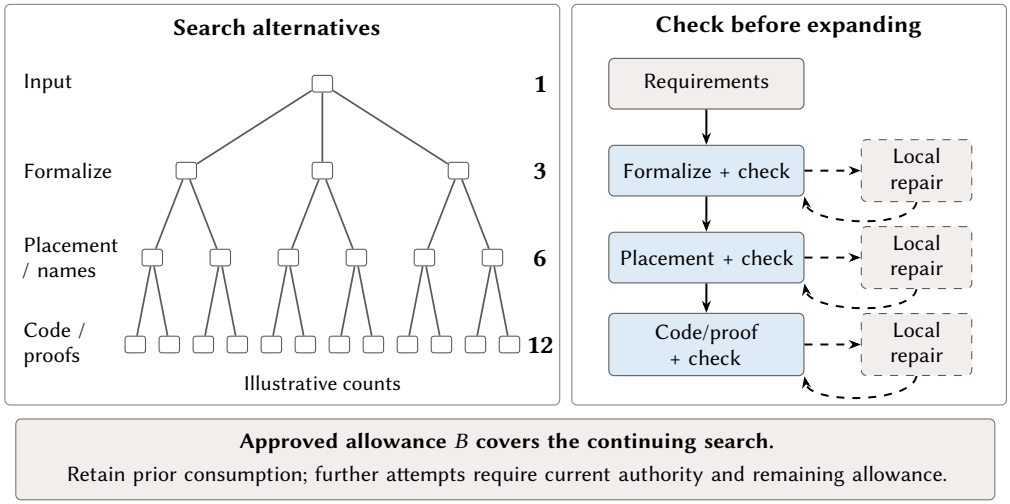


Fig. 10. Illustrative search and local repair, redrawn from Tahat et al. [46, slide 7]. The $1 \rightarrow 3 \rightarrow 6 \rightarrow 12$ expansion shows possible branching. Checks can stop an unsuitable choice and return diagnostics to the stage that produced it. These counts describe the illustration; the experiments report their actual attempts and consumption separately.

Further attempts require current authority, an unexpired deadline, and available allowance after recorded consumption and outstanding reservations. Unresolved usage blocks further model calls. A stopped branch retains its attempted artifacts, diagnostics, and charges. An implementation repair can continue within current authority; a change to requirements, assumptions, scope, or limits returns to human review. A current failed use of the same rule and learning plan can supply feedback for rule revision. An authorized extension preserves earlier consumption across the continuing campaign.

E Requirement Coverage and Evidence Links

This appendix supports the fidelity claims in Section 6. The anonymous extract `evidence/coverage-method-audit.json` records the selected source identifiers, full SHA-256 fingerprints, and evidence links. Its service-source fingerprints match the SNG study freeze. The extract contains selected facts; the complete execution archive remains separate [8, 9].

Producer/Consumer and SNG retain requirement mappings in case-specific audit tables linked to their source, contract, and check records. Table 20 follows one relation in the SNG audit. The campaign records also retain attempts, resource use, assumptions, and review decisions.

Table 20. One recorded SNG requirement-to-evidence trace. Identifiers refer to the accompanying extract.

Link	Recorded contents
English	`Req_R_2.a`: a Restricted payload above 100 becomes 100 (E1, line 38).
Model	The named guarantee `Req_R_2_a_Clamp_Above_100` occurs in the software model (M1, line 90).
Contracts	Gate preserves the Restricted message; Filter requires the present Restricted output to contain 100 (C1, C2).
Evidence	Saved Gate and Filter crate proofs pass. Audit A1 links the contracts to the checked compute methods (P1, P2).
Review	Approval H1 covers the initialization premise and repair. The component relation passes under its premises. `POWERUSERS-1` approved release of the recorded engineering scope on 8 November; system occurrence correspondence remains a separate obligation.

The example assumes empty outgoing slots at dispatch entry and relies on generated external input/output interfaces. The approved initialization premise has supporting factory/composition proofs; correspondence with the actual wrapper and physical transport remains unresolved. The audit retains these limits alongside the passing component relation.

The P1 and P2 proof receipts bind their respective Rust application files, C1 and C2. Their recorded SHA-256 fingerprints match the corresponding entries in the build and observation inventories in `evidence/evidence-audit.json`. These matches identify the application files across checks and builds. The separate runtime evidence measures boot and receiver liveness.

The controller fingerprints declared input-file bytes and resolved configuration. Stored records are rechecked against their hashes; architecture records also identify the tool and comparison procedure. Review fingerprints cover evidence, scope, exclusions, and blockers. Comparing these values during retrieval or a protected action detects changed contents. The linked proposal, check, and decision records identify the responsible agent or human for each reviewed version.

Human review remains explicit for non-behavioral requirements, including timing and deployment, and for assumptions or exclusions. An exclusion records its reason, revisit condition, and approval against the assessed evidence; acceptance requires current passing included checks and current approval. Technical claims still require the applicable analysis, proof, test, or observation. For this SNG case, timing guarantees and full system correspondence remain open.

The workbench also provides optional structured requirement-mapping and proof-unit services. Their configuration fields, `requirement_ledger` and `unit_registry`, are unset in the inspected PC/SNG model runs; these cases use the audit tables described above. A mapping result records whether an element is located. A proof-unit result additionally binds the run, configuration, input files, unit, and checked element. The accompanying extract preserves these implementation details for reproduction.

F Review and Retrieval in the Interface

F.1 Campaign Archive and Review Interface

The browser and command-line interface retrieve checks, evidence, and decisions from the same persistent campaign archive [8]. From a run, an engineer can locate the failed check, verify the evidence fingerprint, and follow links to the reviewer, rationale, and decision time. The examples below show an approved scope change, the retained failure, and the corresponding retrieval commands.

Earlier budget authorization. The earlier 14M campaign allowance provides a concrete authorization example. Its request asks: "Approve 60 minutes and up to 2M accounted tokens within the existing 14M ceiling, retaining prior charges and the documented best-effort/1M unmetered-failure debit policy?" The requested 2M allowance remains within the cumulative 14M ceiling. If u tokens are already accounted for, full use requires $u + 2\text{M} \leq 14\text{M}$; reservations further reduce the available balance.

The `Campaign` service records extensions against the prior allowance or deadline and preserves consumption. The 1M debit estimates usage when measurement is unavailable. Approval must be recorded separately from this request. The recorded name attributes approval; identity authentication remains external to this record [8].

Scope review. In the synthetic case in Figure 11, A passed and B failed. Approving B's exclusion enabled partial acceptance while retaining the failure, rationale, revisit condition, and option to revoke it. The excluded obligation receives no verification credit. The interface's separate *Defer selected* action keeps B required.

F.2 Retrieval Commands

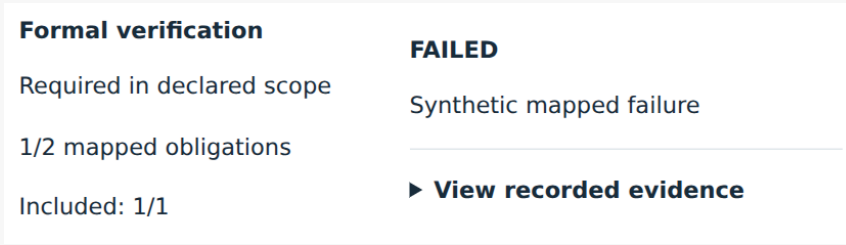
For that case, let R be run `a83f146e…`, H evidence fingerprint `3db914d832ff…`, and P request `610bd646…`. With `./scp-workbench -state-dir STATE`, the schematic queries are

```
trace R
trace R -evidence H
approval P
```

The trace identifies the failed check and H. The second query retrieves the A-pass/B-fail evidence and checks its SHA-256. The review record links it to B's exclusion and request P; the third query retrieves approval, reviewer label, and decision time. A separate acceptance record binds the review fingerprint. The original trace remains `FAILED`.

Signed export uses a bundle location and the expected policy and trusted keys. Three pilot cases check retention across clean service reopening (Section 7); browser-restart behavior remains untested [8].

(a) Verification status and evidence link



(b) Per-requirement results and approval

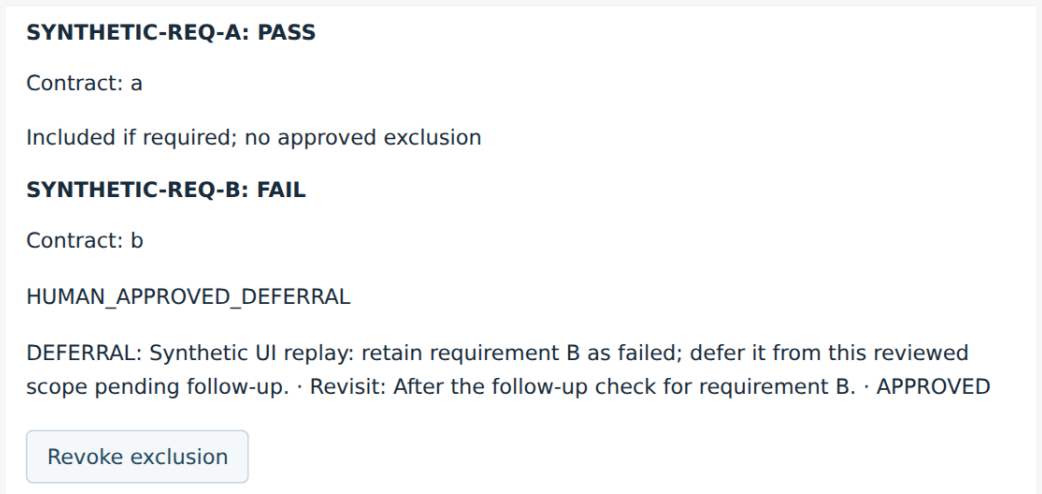


Fig. 11. Review of a synthetic case in the workbench. An approved exclusion leaves one passing obligation in scope (1/1). The original result still shows a failure and 1/2 mapped obligations. The review panel retains the rationale and revisit condition. The evidence link opens the original result.